

Proyecto Final – Avance 1: Documento de Diseño

Sistema de Facturación e Inventario Fidecompro (Proyecto 1) · Versión 10

Estudiante: [Nombre completo] · Carné: [número]

Programación Cliente-Servidor Concurrente · Profesor: Mario Alberto Vargas Montes · Octubre 2026

Control de versiones

Versión	Fecha	Cambios
v1	06/10/2026	Primera versión: clases, 15 historias de usuario, 8 prototipos y diagramas de arquitectura, secuencia y navegación
v2	08/10/2026	Modelo de clases ajustado al material de las semanas 1 a 5 (estilo MultiSports); tabla de temas del curso; hilos, sockets, Swing y BD como arquitectura prevista; historias y prototipo de productos por tipo
v3	08/10/2026	Se agrega esta tabla de control de versiones y la versión en el nombre del archivo
v4	08/10/2026	Historias de usuario en formato de tabla con escenarios de aceptación (Dado que / Cuando / Resultado esperado)
v5	08/10/2026	Se agregan las listas de requerimientos funcionales (10) y no funcionales (8); las secciones siguientes se reenumeran
v6	08/10/2026	Diseño alineado con lo que el profesor pide para el proyecto final: CRUD completo (con eliminar) desde pantallas, ventana del servidor (HU 16 y P9), base de datos MySQL o Derby con su modelo de tablas y plan de demostración para el video final
v7	08/10/2026	Guía al inicio con las tres partes que pide el Avance 1 (historias de usuario, diagrama de clases y bocetos); tabla de pantallas CRUD por clase; nuevo boceto P10 de categorías; nombre del profesor en la portada
v8	08/10/2026	Cada colección según su uso, como en los ejemplos del profesor: <code>List</code> donde importa el orden, <code>Set</code> para advertencias sin repetir, <code>HashMap</code> para usuarios y clientes por id, <code>TreeMap</code> para facturas ordenadas por número y <code>CopyOnWriteArrayList</code> para los clientes conectados al servidor;

Versión	Fecha	Cambios
		nueva tabla «Colecciones: cuál se usa y por qué» en la sección 4.2
v9	08/10/2026	La tabla «Colecciones: cuál se usa y por qué» cita el ejemplo del profesor que respalda cada elección (EjemploList, EjemploSet, EjemploMap, EjemploTreeMap, EjemploCollection) y explica, con EjemploColeccionGenerica y EjemploColeccionNoGenerica, por qué todas las colecciones son genéricas
v10	08/10/2026	La sección 4.1 abre con la vista general del diagrama de clases en una página completa, con su simbología (la misma imagen del diagrama simplificado v4 de draw.io, ya con las colecciones), y menciona los archivos editables de draw.io; la tabla del inicio indica dónde está

1. Introducción

La cadena de venta al por mayor **Fidecompro** necesita una aplicación de escritorio para llevar el inventario de sus productos y emitir facturas a sus clientes. Este documento presenta el diseño de la solución para el Avance 1 del proyecto final: las clases identificadas (con atributos, métodos y relaciones), las funcionalidades expresadas como historias de usuario y los prototipos de la interfaz gráfica.

Contenido del Avance 1. El Avance 1 es conceptual y pide tres partes. Esta tabla indica dónde está cada una; el resto del documento (requerimientos, arquitectura, base de datos y plan de demostración) las complementa.

Parte pedida	Dónde está	Qué contiene
Historias de usuario	Sección 5	16 historias con su rol, funcionalidad, finalidad y escenarios de aceptación
Diagrama de clases	Secciones 4.1 y 4.3	Vista general en una página con su simbología (4.1), diagramas detallados con atributos, métodos y relaciones (4.1 y 4.3); la sección 4.2 describe cada clase
Bocetos (mockups) de las pantallas	Sección 6	Inicio de sesión, menú principal y pantallas CRUD de cada clase (clientes, productos, categorías, usuarios y facturas), más la ventana del servidor

1.1 Requerimientos del enunciado

#	Requerimiento del enunciado	Historias de usuario que lo cubren
R1	Creación de los registros de los clientes	HU 4, HU 5, HU 6
R2	Registro de los productos (varios tipos	HU 7, HU 8, HU 9, HU 10

#	Requerimiento del enunciado	Historias de usuario que lo cubren
	de productos)	
R3	Crear facturas a los clientes	HU 11, HU 12, HU 14, HU 15
R4	Entregar la factura física (archivo con el desglose de pago)	HU 13
R5	Ingreso con usuario y contraseña	HU 1, HU 2, HU 3

1.2 Requisitos del proyecto final

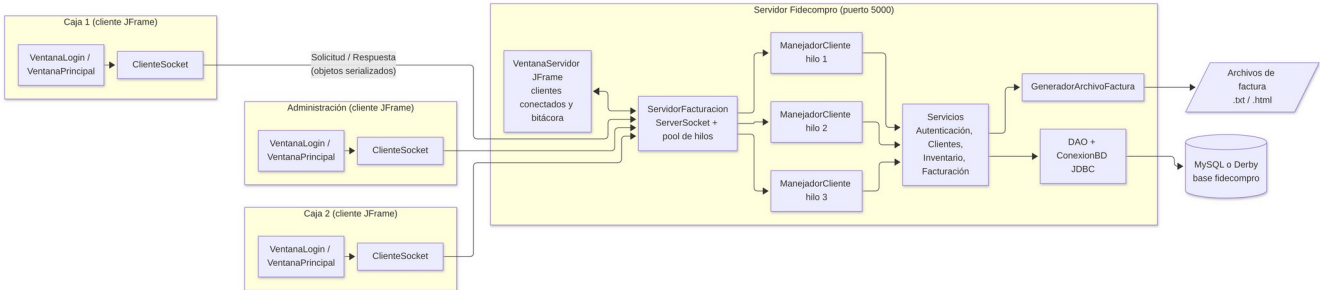
El profesor indicó que el proyecto final debe cumplir cuatro requisitos técnicos y demostrarse en un video. Aunque varios temas todavía no se han visto en el curso, el diseño de este avance ya los contempla para no tener que rehacerlo después:

Requisito del proyecto final	Cómo lo cumple el diseño	Dónde
Cliente-servidor con sockets	Cada caja es una aplicación cliente que se conecta por Socket al ServerSocket del servidor (puerto 5000) e intercambia objetos Solicitud y Respuesta	Secciones 4.3 y 4.4
Hilos	El servidor atiende cada cliente conectado en su propio hilo (ManejadorCliente); las operaciones que modifican datos se sincronizan	Secciones 4.4 y 4.5
Pantallas JFrame	Ventanas Swing para el cliente (P1 a P8 y P10) y una ventana propia del servidor (P9)	Sección 6
Base de datos MySQL o Derby	Todo se guarda en la base de datos por medio de JDBC y clases DAO	Sección 4.6
CRUD completo desde pantallas	Crear, consultar, modificar y eliminar usuarios, clientes, categorías y productos; crear, consultar y anular facturas	Secciones 2 y 8
Varios clientes a la vez	Se pueden abrir varias ventanas cliente al mismo tiempo y el servidor las atiende en paralelo	HU 12, HU 16
Video final	Guion de demostración de todas las funcionalidades	Sección 8

1.3 Alcance y decisiones técnicas

- **Lenguaje y entorno:** Java 17, proyecto Maven en NetBeans IDE, interfaz gráfica con **Java Swing** (ventanas **JFrame** diseñadas en NetBeans). No se usa ningún framework; todo el código se escribe desde cero.
- **Base del diseño:** el modelo de clases (sección 4) aplica lo visto en las semanas 1 a 5: herencia, clases abstractas, interfaces, polimorfismo, colecciones y excepciones propias. Hilos, sockets, pantallas **JFrame** y base de datos aún no se han visto en el curso; se incluyen como **arquitectura prevista**, porque son obligatorios en el proyecto final, y se detallarán en los siguientes avances.

- **Arquitectura cliente-servidor:** un **servidor** central atiende a varias cajas (clientes JFrame) al mismo tiempo mediante **sockets TCP**. Cada conexión es atendida por su propio hilo (ManejadorCliente) dentro de un pool de hilos. El servidor tiene su propia ventana (VentanaServidor) para iniciarlo, detenerlo y ver los clientes conectados y cada operación que atiende.
- **Concurrencia:** cuando dos vendedores facturan el mismo producto a la vez, el servidor valida y descuenta el stock dentro de un bloque **synchronized**, de modo que nunca se venda más inventario del que existe. El número de factura sale del contador **static Factura.ultimoNumero**, que se incrementa dentro de un método **synchronized** para que dos facturas nunca reciban el mismo número. Las demás operaciones que modifican datos (crear, modificar y eliminar) también se sincronizan en el servidor. En el cliente, las consultas largas se ejecutan con **SwingWorker** para no congelar la ventana, y cada pantalla tiene el botón *Actualizar lista* para ver los cambios hechos desde otras cajas.
- **Persistencia:** base de datos **MySQL** accedida con **JDBC** (API estándar de Java) a través de clases DAO. Como alternativa se puede usar **Apache Derby**, la otra base de datos que se verá en el curso: las sentencias SQL son las mismas y solo cambia la URL de conexión en **ConexionBD**.
- **Eliminar sin perder historial:** usuarios, clientes, categorías y productos se pueden eliminar desde su pantalla. Si un registro ya está en una factura (un cliente con compras, por ejemplo), el sistema no lo borra para no dañar el historial; lo desactiva y lo explica al usuario.
- **Factura física:** el servidor genera un archivo **.txt** (y opcionalmente **.html**) con el desglose completo de la factura, que el usuario guarda o imprime.
- **Roles:** **ADMINISTRADOR** (gestiona usuarios, catálogo e inventario, puede anular facturas) y **VENDEDOR** (registra clientes y emite facturas).



Arquitectura general de la solución

2. Lista de requerimientos funcionales

Los requerimientos funcionales describen lo que el sistema debe hacer. Cada uno indica las historias de usuario que lo detallan (sección 5).

#	Requerimiento	Descripción	Prioridad
1	El sistema debe permitir crear, consultar, modificar y eliminar usuarios, y que ingresen con usuario y contraseña.	Solo el personal autorizado puede usar el sistema. Cada usuario tiene un rol (administrador o vendedor) que define a qué módulos entra. Un usuario con facturas emitidas no se elimina para no perder el historial; en ese caso se desactiva. Es el primer requerimiento a implementar porque todas las demás funciones dependen de saber quién está trabajando.	Alta

#	Requerimiento	Descripción	Prioridad
		Historias: HU 1, HU 2, HU 3.	
2	El sistema debe permitir crear, consultar, modificar y eliminar clientes.	Los clientes son la base de la facturación: cada factura se emite a uno de ellos. Incluye tipo y número de identificación, nombre o razón social, teléfono, correo y dirección. Un cliente con facturas no se puede eliminar; en ese caso se desactiva y conserva su historial. Historias: HU 4, HU 5, HU 6.	Alta
3	El sistema debe permitir crear, consultar, modificar y eliminar categorías de productos.	Las categorías (abarrotes, bebidas, limpieza, higiene...) ordenan el catálogo y facilitan encontrar productos al facturar. Una categoría con productos no se puede eliminar. Historia: HU 7.	Media
4	El sistema debe permitir crear, consultar, modificar y eliminar productos de distintos tipos.	La cadena maneja varios tipos de productos (abarrotes, bebida y artículo del hogar), cada uno con datos propios y su propio porcentaje de IVA. Incluye código, nombre, valor de compra, valor de venta, existencias y stock mínimo. Un producto que ya aparece en facturas no se elimina; se desactiva. Historia: HU 8.	Alta
5	El sistema debe permitir registrar entradas de mercadería y ajustes de existencias.	Mantiene el inventario del sistema igual al de bodega. Cada ajuste indica su motivo y nunca puede dejar existencias en negativo. Historia: HU 9.	Alta
6	El sistema debe alertar sobre los productos que están por debajo del stock mínimo.	Permite reabastecer a tiempo y evitar ventas perdidas. La alerta aparece en el menú principal y en una lista ordenada de menos a más existencias. Historia: HU 10.	Media
7	El sistema debe permitir crear facturas a los clientes con uno o varios productos.	Calcula subtotal, descuento, IVA por línea según el tipo de producto y total; asigna un número consecutivo único y descuenta las existencias al emitir. Es el proceso central del negocio. Historias: HU 11, HU 12.	Alta
8	El sistema debe generar la factura física en un archivo con el desglose de pago.	El enunciado pide entregar la factura física. Se genera un archivo con los datos de la empresa, del cliente, cada línea, subtotal, descuento, IVA, total y método de pago, que se puede imprimir o enviar. Historia: HU 13.	Alta
9	El sistema debe permitir consultar el historial de facturas por fechas, cliente y estado.	Permite dar seguimiento a las ventas y conocer el total vendido en un periodo, contando solo las facturas emitidas. Historia: HU 14.	Media
10	El sistema debe permitir anular facturas emitidas.	Corrige ventas registradas por error. Al anular, la factura queda en estado ANULADA y las existencias de cada línea vuelven al inventario. Solo el administrador puede hacerlo. Historia: HU 15.	Media
11	El servidor debe tener su propia ventana para iniciarlo, detenerlo y ver qué clientes están conectados y qué operaciones atiende.	Permite comprobar que el servidor atiende a varias cajas a la vez, cada una en su propio hilo, y que cada operación de las pantallas llega a la base de datos. Historia: HU 16.	Alta

3. Lista de requerimientos no funcionales

Los requerimientos no funcionales describen cómo debe comportarse el sistema y las condiciones técnicas que debe cumplir.

#	Requerimiento	Descripción	Prioridad
1	El sistema debe desarrollarse en	Es una condición del enunciado del proyecto: todo el código se	Alta

#	Requerimiento	Descripción	Prioridad
	Java 17 con NetBeans, sin frameworks ni código previo.	escribe desde cero, usando solo las bibliotecas estándar de Java (Swing, sockets, hilos y JDBC).	
2	El sistema debe contar con pantallas gráficas de escritorio en español hechas con JFrame.	Las ventanas del cliente y del servidor se construyen con Java Swing (JFrame) en NetBeans. Siguen un mismo estilo, muestran mensajes claros y marcan los campos con error, para que el personal de las cajas las use sin capacitación extensa. Todo el CRUD se hace desde las pantallas.	Alta
3	El sistema debe funcionar con una arquitectura cliente-servidor por sockets.	Se pueden abrir varias ventanas cliente al mismo tiempo, en la misma computadora o en varias, y todas se conectan por sockets TCP a un servidor central que guarda la información. Así todas las cajas trabajan sobre los mismos datos de clientes, productos y facturas.	Alta
4	El sistema debe soportar varios clientes haciendo operaciones al mismo tiempo sin generar inconsistencias.	El servidor atiende cada conexión en un hilo distinto. Las operaciones que modifican datos (crear, modificar, eliminar, descontar existencias y numerar facturas) se sincronizan, para que nunca se venda más producto del que hay ni se repita un número de factura. Cada pantalla tiene el botón "Actualizar lista" para ver los cambios hechos desde otras cajas.	Alta
5	El sistema debe proteger el acceso y la información de los usuarios.	Las contraseñas se guardan cifradas, el ingreso se bloquea por un minuto tras tres intentos fallidos y cada rol solo ve los módulos que le corresponden.	Alta
6	El sistema debe almacenar la información en una base de datos MySQL o Derby.	Usuarios, clientes, categorías, productos y facturas se guardan en MySQL (o en Apache Derby, la otra base de datos que se verá en el curso) a través de JDBC, de forma ordenada y sin perderse al cerrar la aplicación. Solo la clase ConexionBD cambia si se pasa de una a otra.	Alta
7	El sistema debe controlar los errores sin dejar datos incompletos.	Los errores se manejan con excepciones propias y mensajes claros. Una operación que falla (por ejemplo, una factura sin stock suficiente) no deja cambios a medias en el inventario.	Alta
8	El sistema debe responder rápido y sin congelar la pantalla.	Las operaciones frecuentes, como buscar un cliente o emitir una factura, deben responder en menos de 2 segundos. Las consultas largas se ejecutan en segundo plano para que la ventana siga respondiendo.	Media

4. Entidades y clases

El modelo de clases sigue la forma de trabajo vista en las semanas 1 a 5 del curso, en particular la práctica integradora MultiSports de la semana 5:

Concepto del curso	Dónde se aplica en Fidecompro
Clase abstracta con subclases que sobrescriben un método (semanas 2 y 3)	Producto – Abarrote, Bebida, ArtículoHogar; Persona – Usuario, Cliente
Interface terminada en <i>-able</i> con <code>mostrarInformacion()</code> (semanas 3 y 5)	Mostrable, implementada por personas, productos, facturas y detalles
Polimorfismo: distintos objetos responden distinto al mismo mensaje (semana 3)	Cada tipo de producto responde <code>obtenerPorcentajeImpuesto()</code> a su manera
Atributos <code>static</code> e IDs autoincrementales (semanas 1 y 5)	<code>idAutoIncremental</code> en Producto, Categoria, Cliente y Usuario; <code>Factura.ultimoNumero</code>
Constantes <code>final</code> (semana 1, ejemplo del IVA)	<code>Producto.IVA_GENERAL = 0.13</code> y <code>Abarrote.IVA_CANASTA_BASICA = 0.01</code>
Composición y agregación con colecciones genéricas	<code>Factura contiene</code> sus <code>DetalleFactura</code> ; <code>Inventario</code>

Concepto del curso	Dónde se aplica en Fidecompro
(semanas 1 y 5)	agrupa Categoria y Categoria agrupa Producto
Colecciones List , Set y Map (HashMap , TreeMap) declaradas por su interface, cada una según su uso (semana 4)	List en detalles, productos y categorías; Set en advertencias; Map en los registros de usuarios, clientes y facturas (tabla «Colecciones» en la sección 4.2)
Métodos agregarX / editarX / eliminarX / mostrarX por id (semana 5)	Inventario , Categoria , RegistroClientes , RegistroUsuarios , RegistroFacturas
Enumeraciones para catálogos cerrados (semana 1)	Rol , TipoIdentificacion , MetodoPago , EstadoFactura
Comparable y Comparator (semana 4)	Producto implements Comparable<Producto> (orden por nombre) y ComparadorPorExistencias
Excepciones propias y try/catch/finally (semana 4)	StockInsuficienteException , CredencialesInvalidasException , RegistroNoEncontradoException ; IOException al escribir la factura

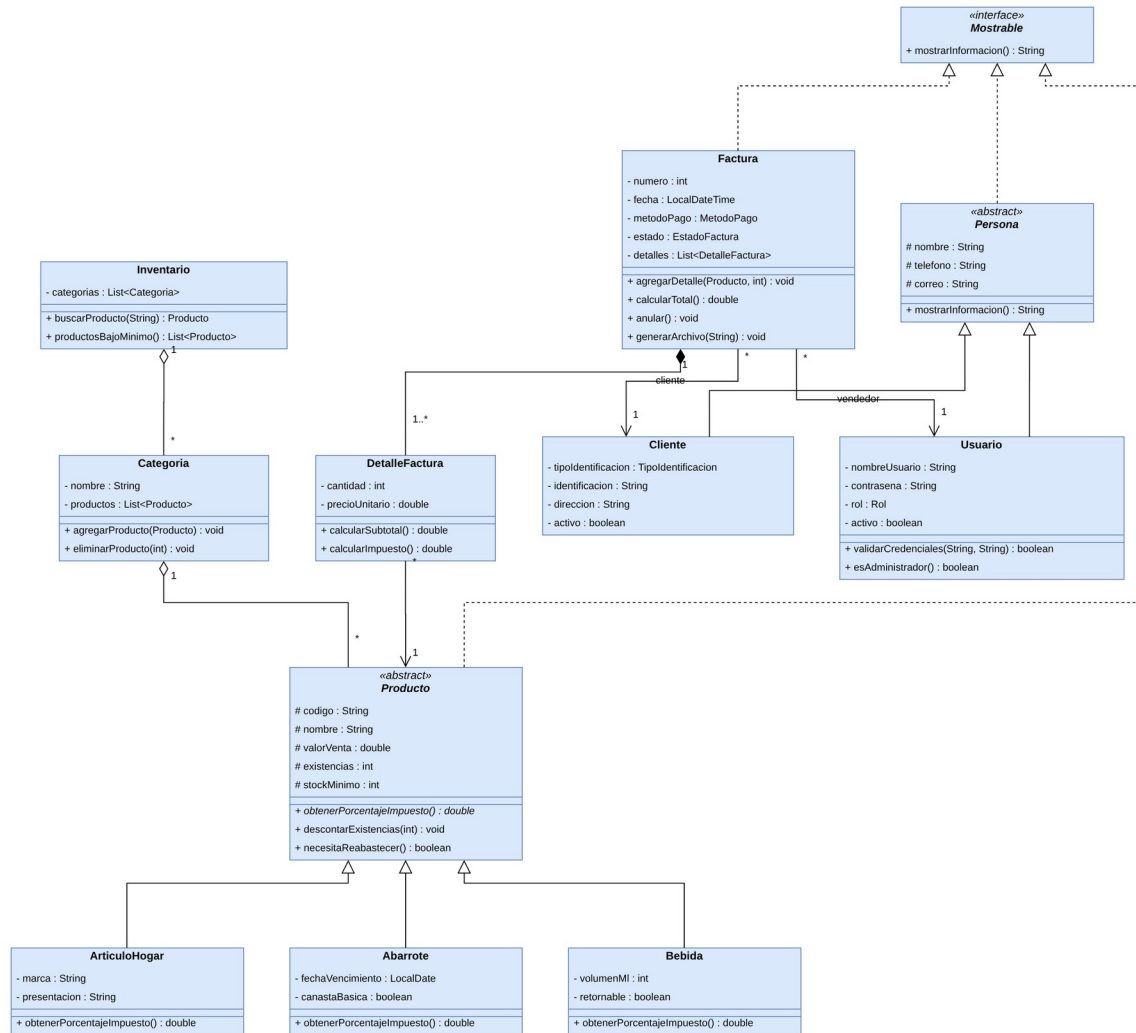
Las clases se organizan en paquetes de un proyecto Maven de NetBeans:

Paquete	Contenido
<code>com.fidecompro.modelo</code>	Entidades, enumeraciones, interface Mostrable y clases que administran colecciones (Inventario , Registro ...)
<code>com.fidecompro.excepciones</code>	Excepciones propias del negocio
<code>com.fidecompro.servidor</code>	Servidor de sockets, hilos de atención y servicios (arquitectura prevista, sección 4.3)
<code>com.fidecompro.datos</code>	Conexión a la base de datos y clases DAO (arquitectura prevista)
<code>com.fidecompro.cliente</code>	Conexión del cliente y ventanas Swing (arquitectura prevista)

4.1 Diagramas de clases del modelo de dominio

Primero se muestra una vista general en una sola página con las 12 clases principales, sus atributos y métodos más importantes, sus relaciones y una simbología que explica cada símbolo del diagrama (visibilidad, static, abstracto, multiplicidad, tipos de flecha y colecciones). Después, el detalle completo en dos diagramas para que se lea con claridad. Todos los diagramas de clases se entregan también en draw.io para editarlos (**Avancel_Diagrama_de_Clases_Simplificado_v4.drawio** y **Avancel_Diagrama_de_Clases_v4.drawio**).

Vista general del modelo de dominio



Simbología del diagrama

Partes de cada clase

Nombre	Arriba va el nombre de la clase; en el medio, sus atributos; abajo, sus métodos.
nombre : Tipo	Atributo y su tipo de dato. Ejemplo: existencias : int.
metodo(...): Tipo	Método, los datos que recibe y el tipo de dato que devuelve (void = no devuelve nada).

Visibilidad

+	Público: cualquier clase lo puede usar.
-	Privado: solo lo usa la misma clase.
#	Protegido: lo usan la clase y sus subclases.

Otros símbolos

subrayado	Static: pertenece a la clase y no a cada objeto (por ejemplo, el contador de ids).
cursiva	Abstracto: la clase no se puede instanciar o el método no tiene código y lo escribe cada subclase.
<<abstract>>	Clase abstracta: sirve de base para otras clases (Persona, Producto).
<<interface>>	Interface: lista de métodos que las clases que la implementan están obligadas a tener.

Multiplicidad (cuántos objetos se relacionan)

1	Exactamente uno.
*	Cero o muchos.
1..*	Uno o muchos (al menos uno).

Ejemplo: Factura * -- 1 Cliente se lee "cada factura es de un cliente y un cliente puede tener muchas facturas".

Relaciones

	Herencia: la subclase es un tipo de la superclase (Usuario es una Persona).
	Implementación: la clase cumple una interface (Producto implementa Mostrable).
	Composición: el todo contiene sus partes y estas no existen sin él (Factura y sus detalles).
	Agregación: el todo agrupa partes que pueden existir por separado (Categoria y sus productos).
	Asociación: una clase conoce y usa a otra (la Factura conoce a su Cliente).

Colecciones

List<Tipo>	Lista: guarda los objetos en orden (las líneas de una factura, los productos de una categoria).
-------------------------	---

En el diagrama completo también hay **Set** (no acepta repetidos) y **Map / TreeMap** (guarda cada objeto con una clave, como el id del cliente o el número de factura).

Diagrama de clases simplificado con simbología

Detalle del modelo de dominio

El primer diagrama detallado cubre el inventario, los tipos de producto y las excepciones; el segundo, las personas y la facturación.

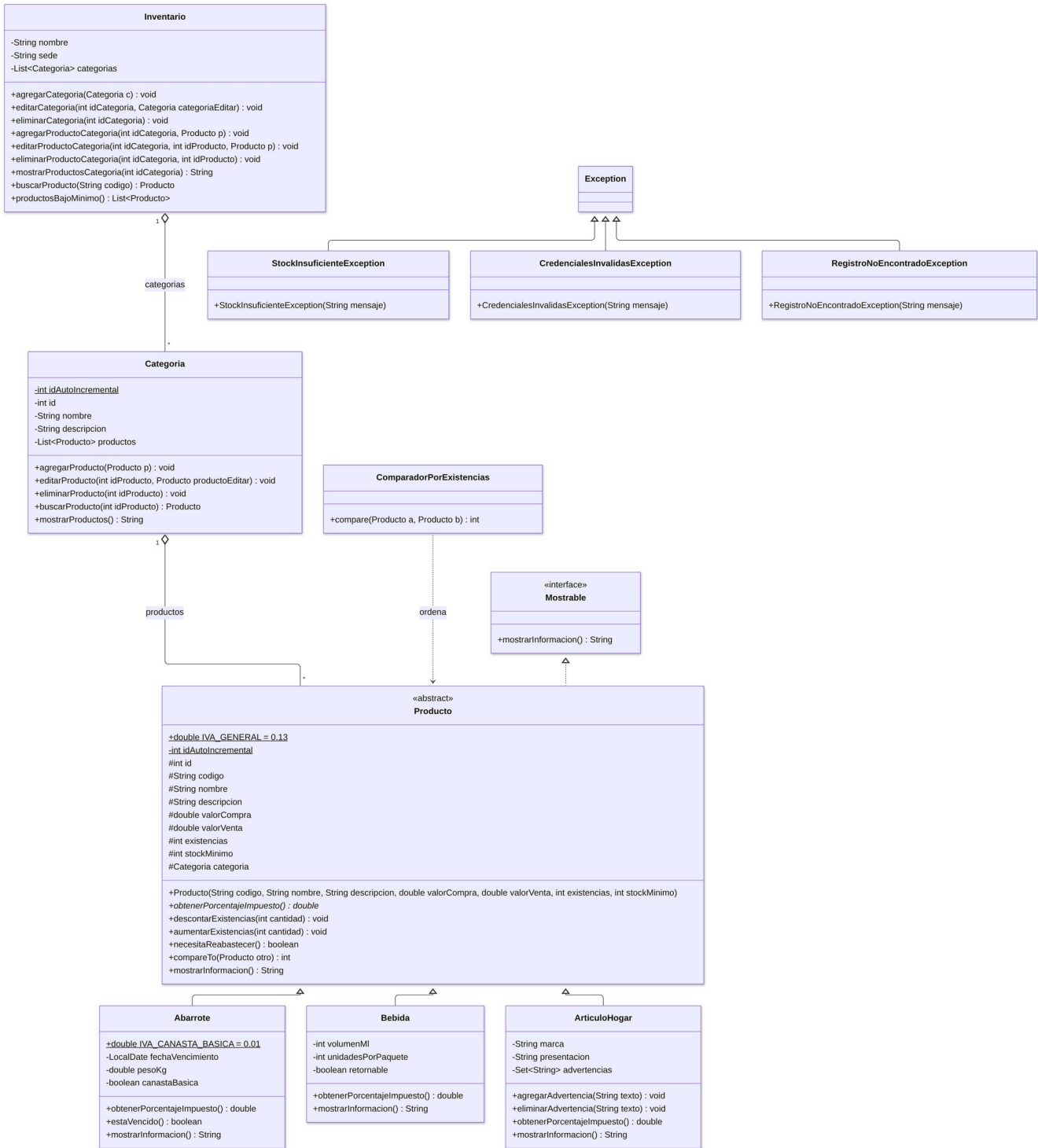


Diagrama de clases: inventario, tipos de producto y excepciones

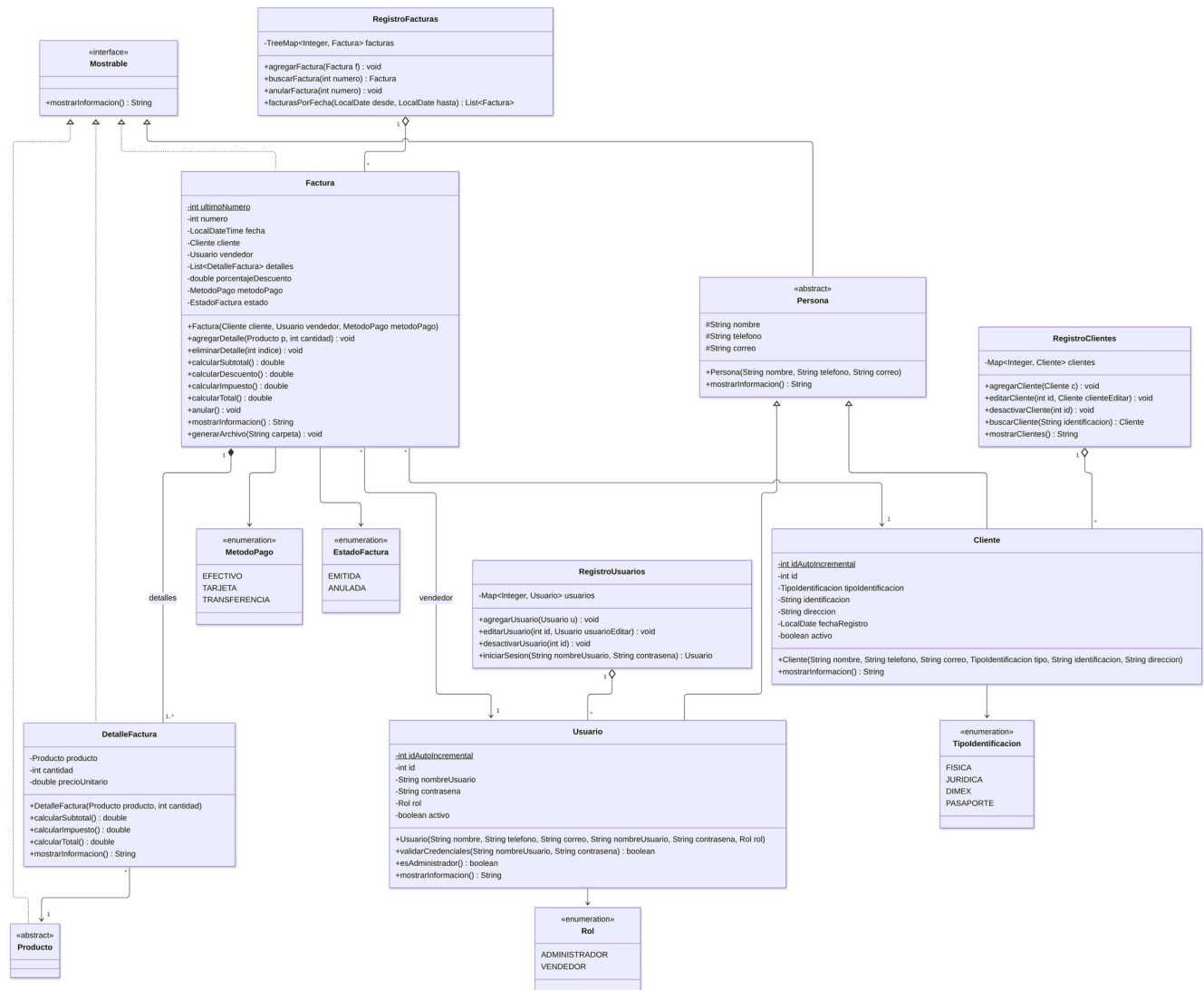


Diagrama de clases: personas y facturación

4.2 Descripción de las clases del dominio

Todas las clases tienen constructor con sus campos, *getters* y *setters*. En las tablas se listan solo los métodos con lógica propia.

Mostrable (interface)

Contrato que obliga a cada objeto a describirse a sí mismo como texto. Se usa para listar en pantalla y para armar la factura física.

Método	Descripción
mostrarInformacion(): String	Devuelve la información del objeto lista para mostrar o imprimir

Persona (abstracta)

Datos comunes de quien usa el sistema y de a quien se le vende. No se instancia directamente. Implementa Mostrable.

Atributo	Tipo	Descripción
nombre	String	Nombre completo o razón social
telefono	String	Teléfono de contacto
correo	String	Correo electrónico

Relaciones: superclase de Usuario y Cliente (herencia jerárquica).

Usuario (extends Persona)

Persona que ingresa al sistema: vendedor o administrador.

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase para asignar el siguiente id
id	int	Identificador asignado en el constructor
nombreUsuario	String	Nombre con el que inicia sesión (único)
contrasena	String	Contraseña de acceso
rol	Rol	ADMINISTRADOR o VENDEDOR
activo	boolean	Un usuario inactivo no puede ingresar

Método	Descripción
validarCredenciales(String, String): boolean	Compara usuario y contraseña con los suyos y revisa que esté activo
esAdministrador(): boolean	Indica si puede entrar a los módulos restringidos
mostrarInformacion(): String	Sobrescribe el de Persona agregando usuario y rol

Relaciones: emite muchas Factura (1 a *); lo administra RegistroUsuarios.

Cliente (extends Persona)

Comercio o persona a quien Fidecompro le vende.

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
tipoIdentificacion	TipoIdentificacion	Física, jurídica, DIMEX o pasaporte
identificacion	String	Número de cédula o documento (único)
direccion	String	Dirección de entrega
fechaRegistro	LocalDate	Fecha en que se creó el registro
activo	boolean	Los clientes inactivos no aparecen al facturar

Método	Descripción
mostrarInformacion(): String	Sobrescribe el de <code>Persona</code> agregando identificación y dirección

Relaciones: tiene muchas `Factura` (1 a *); lo administra `RegistroClientes`.

Producto (abstracta)

Representa cualquier artículo que vende la cadena. Como el enunciado indica que Fidecompro maneja **varios tipos de productos**, cada tipo es una subclase, igual que `Pelota`, `Bate` y `EquipoDeAnotacion` en la práctica `MultiSports`. Implementa `Mostrable` y `Comparable<Producto>`.

Atributo	Tipo	Descripción
IVA_GENERAL	static final double	Constante 0.13 (tarifa general del IVA)
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
codigo	String	Código interno único (p. ej. ABR-0012)
nombre, descripcion	String	Datos descriptivos
valorCompra	double	Costo al que Fidecompro compra el producto
valorVenta	double	Precio de venta sin impuesto
existencias	int	Unidades disponibles
stockMinimo	int	Nivel a partir del cual se alerta reabastecer
categoria	Categoria	Categoría a la que pertenece

Método	Descripción
obtenerPorcentajeImpuesto(): double	Abstracto. Cada tipo de producto define su impuesto (polimorfismo)
descontarExistencias(int): void	Resta unidades; lanza <code>StockInsuficienteException</code> si no alcanzan
aumentarExistencias(int): void	Suma unidades (compras, ajustes o facturas anuladas)
necesitaReabastecer(): boolean	<code>existencias <= stockMinimo</code>
compareTo(Producto): int	Orden natural por nombre, para listar el catálogo con <code>Collections.sort</code>
mostrarInformacion(): String	Código, nombre, precio y existencias

Relaciones: superclase de `Abarrote`, `Bebida` y `ArticuloHogar`; agregado en `Categoria` (* a 1); referenciado por `DetalleFactura`.

Abarrote, Bebida y ArtículoHogar (extends Producto)

Subclase	Atributos propios	Impuesto (obtenerPorcentajeImpuesto)	Métodos propios
Abarrote (arroz, frijoles, aceite, café...)	fechaVencimiento: LocalDate, pesoKg: double, canastaBasica: boolean; constante IVA_CANASTA_BASICA = 0.01	1 % si es canasta básica, si no IVA_GENERAL	estaVencido(): boolean
Bebida (refrescos, jugos, agua)	volumenMl: int, unidadesPorPaquete: int, retornable: boolean	IVA_GENERAL	—
ArtículoHogar (limpieza e higiene)	marca: String, presentacion: String, advertencias: Set<String> (sin repetidas)	IVA_GENERAL	agregarAdvertencia(String), eliminarAdvertencia(String)

Las tres sobrescriben `mostrarInformacion()` llamando a `super.mostrarInformacion()` y agregando sus datos propios.

Categoria

Agrupar productos para ordenar el catálogo (Abarrotes, Bebidas, Limpieza, Higiene...), igual que en MultiSports.

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
nombre, descripcion	String	Datos de la categoría
productos	List<Producto>	Productos de la categoría, en orden

Método	Descripción
agregarProducto(Producto): void	Agrega un producto y le asigna esta categoría
editarProducto(int idProducto, Producto): void	Reemplaza los datos del producto con ese id
eliminarProducto(int idProducto): void	Quita el producto; lanza <code>RegistroNoEncontradoException</code> si no existe
buscarProducto(int idProducto): Producto	Busca por id
mostrarProductos(): String	Une el <code>mostrarInformacion()</code> de cada producto

Relaciones: agregación con `Producto` (1 a *); agregada en `Inventario`.

Inventario

Punto central del catálogo de una sede.

Atributo	Tipo	Descripción
nombre	String	Nombre del inventario
sede	String	Sucursal de Fidecompro
categorias	List<Categoria>	Categorías registradas

Método	Descripción
agregarCategoria / editarCategoria / eliminarCategoria	CRUD de categorías por id
agregarProductoCategoria(int idCategoria, Producto)	Agrega un producto en la categoría indicada
editarProductoCategoria / eliminarProductoCategoria	Edita o elimina un producto dentro de una categoría
mostrarProductosCategoria(int idCategoria): String	Lista los productos de una categoría
buscarProducto(String codigo): Producto	Busca en todas las categorías; se usa al facturar
productosBajoMinimo(): List<Producto>	Productos con necesitaReabastecer(), ordenados con ComparadorPorExistencias

ComparadorPorExistencias (implements Comparator<Producto>)

Orden alterno al natural: de menos a más existencias, para mostrar primero lo que urge reabastecer.

Factura

Documento de venta emitido a un cliente. Implementa Mostable.

Atributo	Tipo	Descripción
ultimoNumero	static int	Último consecutivo usado (como Factura.ultimoNumero en el ejemplo de la semana 1)
numero	int	Consecutivo de esta factura (se muestra como FC - 000123)
fecha	LocalDateTime	Fecha y hora de emisión
cliente	Cliente	A quién se factura
vendedor	Usuario	Quién la emite
detalles	List<DetalleFactura>	Líneas de la factura, en el orden en que se agregan
porcentajeDescuento	double	Descuento aplicado al subtotal
metodoPago	MetodoPago	Efectivo, tarjeta o transferencia
estado	EstadoFactura	EMITIDA o ANULADA

Método	Descripción
agregarDetalle(Producto, int): void	Crea la línea; si el producto ya está, suma la cantidad. Lanza StockInsuficienteException
eliminarDetalle(int): void	Quita una línea
calcularSubtotal / calcularDescuento / calcularImpuesto / calcularTotal	Recorren detalles y suman; el impuesto de cada línea lo da su producto (polimorfismo)
anular(): void	Cambia el estado a ANULADA
mostrarInformacion(): String	Arma el desglose completo de la factura como texto

Método	Descripción
generarArchivo(String carpeta): void	Escribe mostrarInformacion() en FC-000123.txt con FileWriter, manejando IOException con try/catch/finally

Relaciones: composición con DetalleFactura (1 a 1..*): las líneas no existen sin su factura. Asociación con Cliente y Usuario (* a 1). La administra RegistroFacturas.

DetalleFactura

Línea de una factura. Implementa Mostrable.

Atributo	Tipo	Descripción
producto	Producto	Producto vendido
cantidad	int	Unidades
precioUnitario	double	Copia del valorVenta al momento de la venta, para conservar el histórico

Método	Descripción
calcularSubtotal(): double	cantidad × precioUnitario
calcularImpuesto(): double	subtotal × producto.obtenerPorcentajeImpuesto()
calcularTotal(): double	Subtotal más impuesto
mostrarInformacion(): String	Una línea del desglose: cantidad, descripción, precio y total

RegistroUsuarios, RegistroClientes y RegistroFacturas

Clases que administran las colecciones, con el mismo patrón de Inventario. Usuarios y clientes se guardan en un Map cuya clave es su id, y las facturas en un TreeMap cuya clave es el número, así buscar, editar o eliminar no necesita recorrer toda la colección.

Clase	Atributo	Métodos
RegistroUsuarios	usuarios: Map<Integer, Usuario> (clave: id)	agregarUsuario, editarUsuario, eliminarUsuario, desactivarUsuario, iniciarSesion(nombreUsuario, contraseña) que devuelve el Usuario o lanza CredencialesInvalidasException
RegistroClientes	clientes: Map<Integer, Cliente> (clave: id)	agregarCliente (rechaza identificaciones repetidas), editarCliente, eliminarCliente, desactivarCliente, buscarCliente(identificacion), mostrarClientes
RegistroFacturas	facturas: TreeMap<Integer, Factura> (clave: número, ordenadas)	agregarFactura, buscarFactura(numero),

Clase	Atributo	Métodos
		<code>anularFactura(numero)</code> (devuelve las existencias al inventario), <code>facturasPorFecha(desde, hasta)</code> , <code>tieneFacturasCliente(idCliente)</code> y <code>tieneFacturasUsuario(idUsuario)</code> (se consultan antes de eliminar)

Excepciones propias (extends Exception)

Excepción	Cuándo se lanza
<code>StockInsuficienteException</code>	Se intenta facturar o descontar más unidades de las que hay
<code>CredencialesInvalidasException</code>	Usuario o contraseña incorrectos, o usuario inactivo
<code>RegistroNoEncontradoException</code>	Se busca, edita o elimina un id o identificación que no existe

Enumeraciones

- Rol: ADMINISTRADOR, VENDEDOR.
- TipoIdentificacion: FISICA, JURIDICA, DIMEX, PASAPORTE.
- MetodoPago: EFECTIVO, TARJETA, TRANSFERENCIA.
- EstadoFactura: EMITIDA, ANULADA.

Colecciones: cuál se usa y por qué

Como en los ejemplos de colecciones del profesor (semana 4), no todo se guarda en una lista: cada colección se elige según lo que se necesita (orden, duplicados o búsqueda por clave). Las variables se declaran por su interface y se crean con una implementación, igual que `List<String> lista = new ArrayList<>()` en `EjemploList`, por ejemplo `List<DetalleFactura> detalles = new ArrayList<>()` o `Map<Integer, Cliente> clientes = new HashMap<>()`. Todas son genéricas, como en `EjemploColeccionGenerica`: así no se necesita conversión de tipo (cast) al sacar un elemento, que es el problema que muestra `EjemploColeccionNoGenerica`.

Colección	Dónde se usa	Por qué	Ejemplo del curso
<code>List (ArrayList)</code>	<code>Factura.detalles</code>	Las líneas se muestran en el orden en que se agregan y se quitan por posición (<code>eliminarDetalle(indice)</code>)	<code>EjemploList</code> : mantiene el orden de inserción y permite <code>get</code> y <code>remove</code> por índice
<code>List (ArrayList)</code>	<code>Categoria.productos</code> , <code>Inventario.categorias</code> y los resultados de <code>listarProductos()</code> , <code>productosBajoMinimo()</code> y <code>facturasPorFecha()</code>	Se recorren y se muestran en orden, y se ordenan con <code>Collections.sort</code> usando <code>Comparable</code> (por nombre) o <code>Comparator</code> (por existencias)	<code>EjemploList</code> y <code>EjemploCollection</code> (recorrido con <code>for</code> mejorado)
<code>Set (HashSet)</code>	<code>ArticuloHogar.advertencias</code>	Una advertencia no debe salir dos veces en el producto; el <code>Set</code> no acepta elementos repetidos	<code>EjemploSet</code> : <code>HashSet</code> ignora el elemento duplicado
<code>Map (HashMap)</code>	<code>RegistroUsuarios.us</code>	El CRUD es por id:	<code>EjemploMap</code> : pares clave-

Colección	Dónde se usa	Por qué	Ejemplo del curso
	uarios, RegistroClientes.cl ientes	get(id), put(id, ...) y remove(id) llegan directo al registro sin recorrer la colección, y no puede haber dos con el mismo id (igual que la llave primaria en la base de datos)	valor con put, get, containsKey y remove
TreeMap	RegistroFacturas.facturas	Se busca la factura por su número y, además, el TreeMap las mantiene ordenadas por número: el historial sale en orden y firstKey()/lastKey() dan la primera y la última	EjemploTreeMap: claves ordenadas, primera y última entrada y rangos con subMap
CopyOnWriteArrayList	ServidorFacturacion .clientesConectados	Varios hilos agregan y quitan clientes conectados mientras la ventana del servidor recorre la lista; esta lista es segura para hilos	No está en los ejemplos: corresponde a hilos, que el curso verá más adelante

En el servidor, cuando varios hilos `ManejadorCliente` usen los mismos registros a la vez, los `HashMap` se crean como `ConcurrentHashMap` o se protegen con métodos `synchronized`, como ya se hace en `agregarFactura` y `anularFactura`.

4.3 Diagrama de clases cliente-servidor

Este diagrama es la **arquitectura prevista** para los siguientes avances, cuando el curso cubra hilos, sockets, pantallas `JFrame` y bases de datos. Muestra las clases que harán funcionar la aplicación en red y de forma concurrente sobre el modelo de la sección 4.1: las ventanas del cliente y del servidor, la conexión por sockets, el servidor con sus hilos, los servicios y el acceso a datos. La enumeración `TipoOperacion` incluye las cuatro operaciones del CRUD de cada entidad.

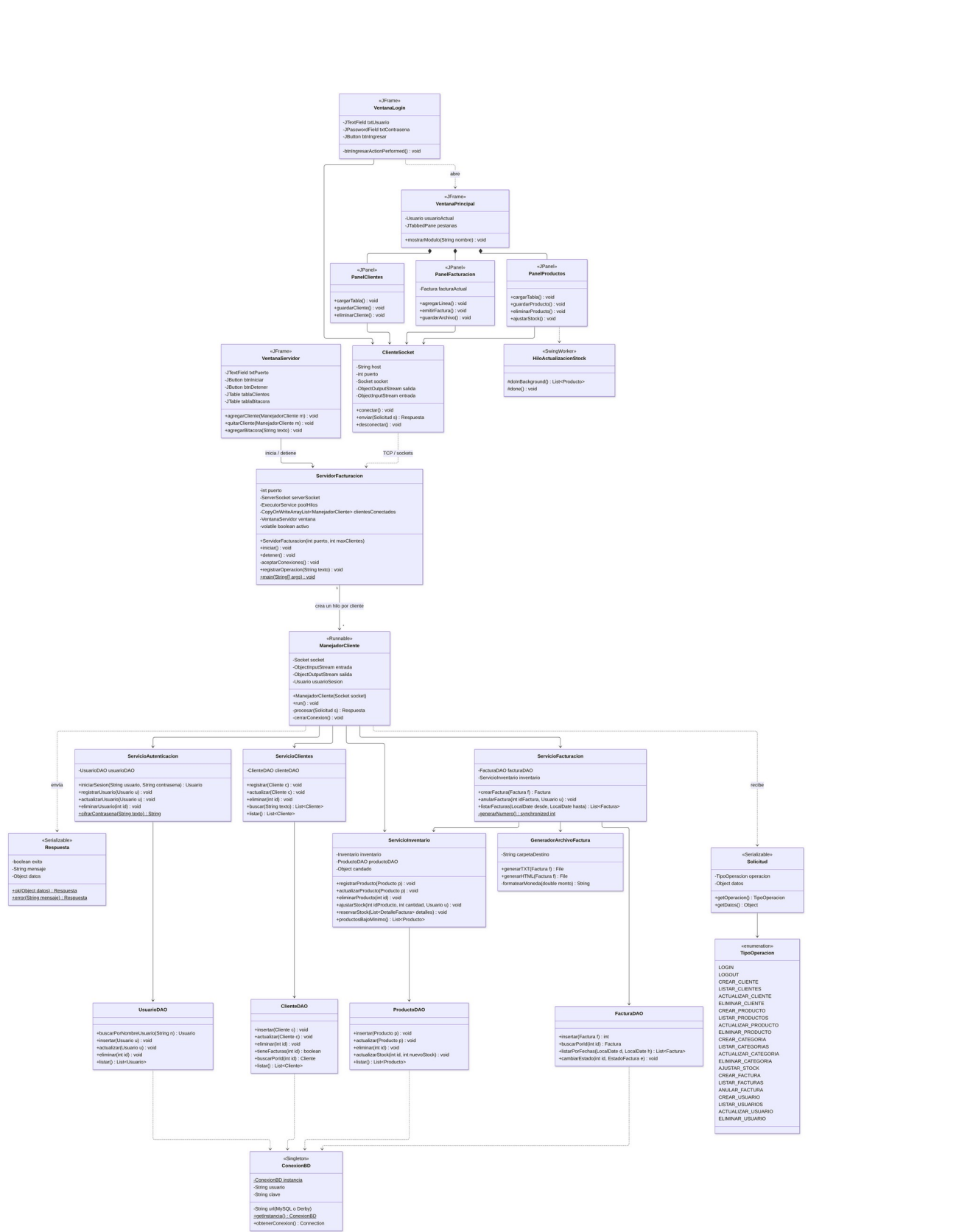


Diagrama de clases cliente-servidor

4.4 Descripción de las clases cliente-servidor

Clase	Responsabilidad	Métodos principales	Relaciones
VentanaServidor (JFrame)	Pantalla del servidor: puerto, botones Iniciar y Detener, tabla de clientes conectados y bitácora de operaciones	agregarCliente(), quitarCliente(), agregarBitacora()	Inicia y detiene ServidorFacturacion
ServidorFacturacion	Abre un <code>ServerSocket</code> en el puerto 5000 y acepta conexiones; cada conexión se entrega a un pool de hilos (<code>ExecutorService</code>). Lleva los clientes conectados en una <code>CopyOnWriteArrayList</code> (segura para hilos) y avisa a la ventana de cada operación	iniciar(), detener(), aceptarConexiones(), registrarOperacion(), main()	Crea muchos ManejadorCliente (1a *)
ManejadorCliente (implementa <code>Runnable</code>)	Atiende a un cliente conectado en su propio hilo: lee <code>Solicitud</code> , llama al servicio correspondiente y responde con <code>Respuesta</code> . Guarda el usuario de la sesión para validar permisos	run(), procesar(Solicitud), cerrarConexion()	Usa los cuatro servicios
Solicitud / Respuesta	Objetos serializables que viajan por el socket. La solicitud indica la operación (<code>TipoOperacion</code>) y los datos; la respuesta indica éxito, mensaje y datos	getOperacion(), ok(), error()	Solicitud usa TipoOperacion
ServicioAutenticacion	Valida credenciales, bloquea usuarios inactivos y registra usuarios nuevos	iniciarSesion(), registrarUsuario(), actualizarUsuario(), eliminarUsuario(), cifrarContraseña()	Usa <code>UsuarioDAO</code>
ServicioClientes	Reglas de negocio de clientes (identificación única, campos obligatorios, no eliminar clientes con facturas)	registrar(), actualizar(), eliminar(), buscar(), listar()	Usa <code>ClienteDAO</code>
ServicioInventario	Catálogo y stock. Sección crítica: <code>reservarStock()</code> y <code>ajustarStock()</code> se sincronizan sobre un mismo candado para que dos hilos no descuenten el mismo producto a la vez	registrarProducto(), actualizarProducto(), eliminarProducto(), ajustarStock(), reservarStock(), productosBajoMinimo()	Usa <code>Inventario y ProductoDAO</code>
ServicioFacturacion	Arma y guarda la factura, reserva el stock, asigna el consecutivo de Factura en un método <code>synchronized</code> y genera el archivo	crearFactura(), anularFactura(), listarFacturas()	Usa <code>ServicioInventario</code> , <code>FacturaDAO</code> , <code>GeneradorArchivoFactura</code>
GeneradorArchivoFactura	Escribe la factura física con	generarTXT(),	Usado por

Clase	Responsabilidad	Métodos principales	Relaciones
	su desglose en .txt o .html usando FileWriter/PrintWriter	generarHTML()	ServicioFacturacion
ConexionBD (Singleton)	Centraliza la URL y credenciales de la base de datos (MySQL, o Derby si se cambia la URL) y entrega conexiones JDBC	getInstancia(), obtenerConexion()	Usada por todos los DAO
UsuarioDAO, ClienteDAO, ProductoDAO, FacturaDAO	Ejecutan las sentencias SQL (PreparedStatement) de cada entidad	insertar(), listar(), buscarPorId(), actualizar(), eliminar() (CRUD)	Dependen de ConexionBD
ClienteSocket	Del lado del cliente: abre el socket hacia el servidor y envía/recibe objetos	conectar(), enviar(Solicitud), desconectar()	Usado por todas las ventanas
VentanaLogin (JFrame)	Pide usuario y contraseña; si son válidos abre la ventana principal	btnIngresarActionPerformed()	Abre VentanaPrincipal
VentanaPrincipal (JFrame)	Menú y pestañas de los módulos; oculta los módulos de administrador a los vendedores	mostrarModulo()	Contiene los paneles
PanelClientes, PanelProductos, PanelFacturacion (JPanel)	Pantallas de cada módulo	cargarTabla(), guardar...(), eliminar...(), emitirFactura()	Usan ClienteSocket
HiloActualizacionStock (SwingWorker)	Refresca la tabla de inventario en segundo plano sin congelar la interfaz	doInBackground(), done()	Usado por PanelProductos

4.5 Flujo concurrente de emisión de una factura

El siguiente diagrama de secuencia muestra cómo se emite una factura y dónde se controla la concurrencia.

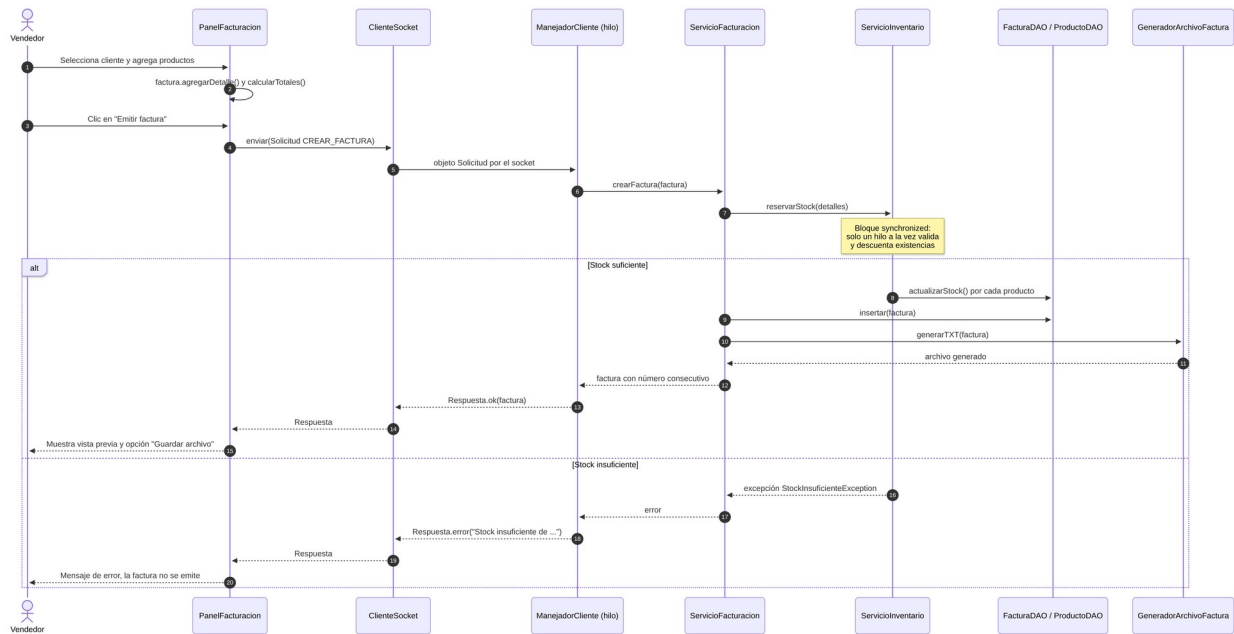
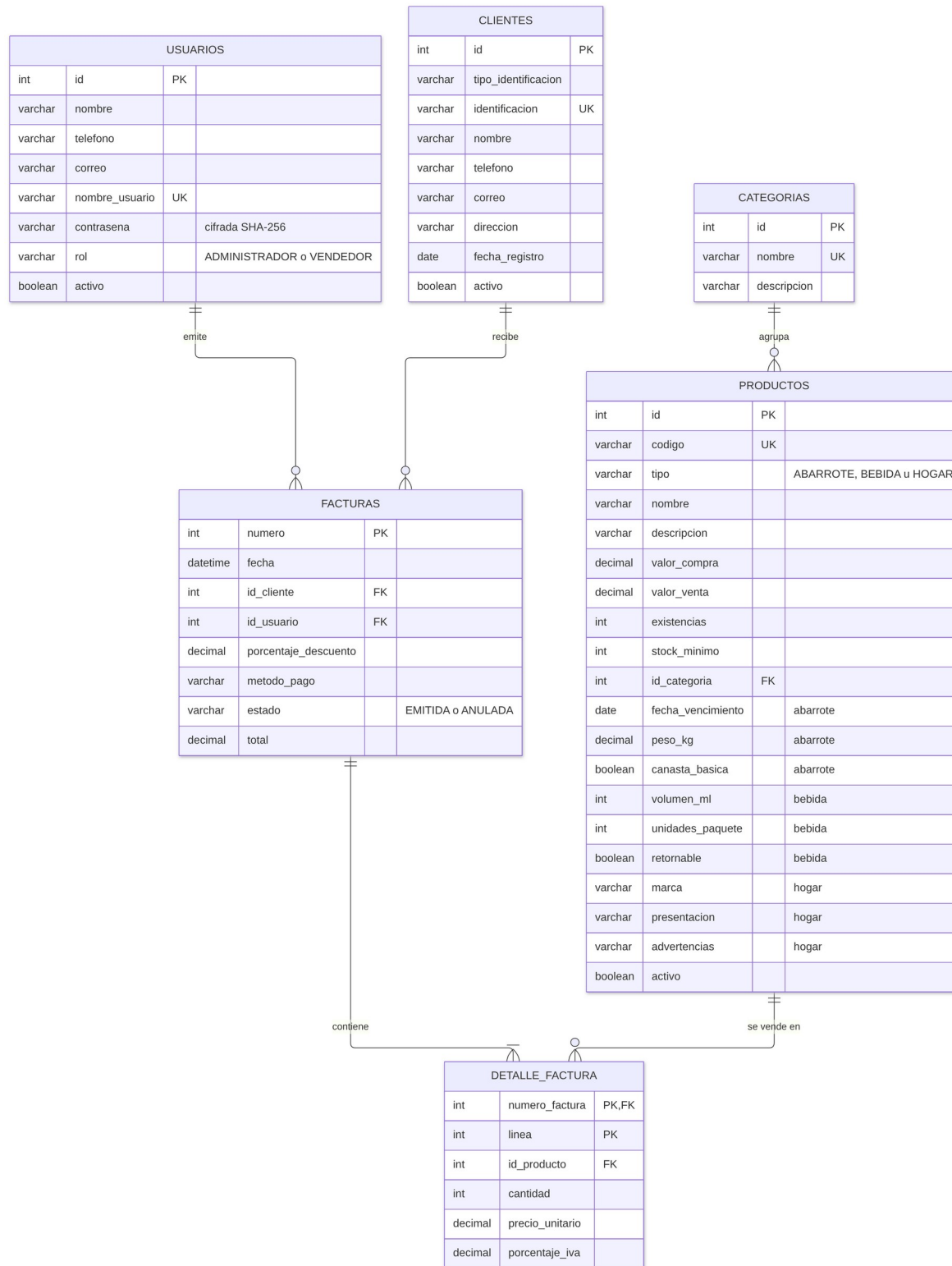


Diagrama de secuencia: emitir factura

4.6 Modelo de la base de datos

La base de datos fidecompro guarda todo lo que se hace desde las pantallas. Se diseña para MySQL y funciona igual en Apache Derby. Los tres tipos de producto se guardan en una sola tabla productos, con una columna tipo y las columnas propias de cada tipo (las que no aplican quedan en NULL).



Modelo de la base de datos

Tabla	Guarda	Llave primaria	Relaciones
usuarios	Cuentas de acceso con su rol	id	Un usuario emite muchas

Tabla	Guarda	Llave primaria	Relaciones
	y contraseña cifrada		facturas
clientes	Clientes de Fidecompro	id	Un cliente tiene muchas facturas
categorias	Categorías del catálogo	id	Una categoría agrupa muchos productos
productos	Productos de los tres tipos, con existencias	id	Pertenece a una categoría; aparece en muchas líneas de factura
facturas	Encabezado de cada factura	numero	Pertenece a un cliente y a un usuario
detalle_factura	Líneas de cada factura con el precio e IVA del momento	numero_factura + linea	Pertenece a una factura y a un producto

5. Historias de usuario

Cada historia sigue el formato *Como un <rol>, necesito <funcionalidad>, con la finalidad de <resultado>* y se acompaña de sus escenarios de aceptación en la forma **Dado que** (contexto), **Cuando** (evento) y **resultado esperado**. Los escenarios cubren el caso exitoso y los errores que el sistema debe controlar.

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
HU 1	Como un usuario (vendedor o administrador)	Necesito ingresar a la aplicación con mi nombre de usuario y contraseña	Con la finalidad de que solo el personal autorizado pueda facturar y modificar el inventario	1	Inicio de sesión exitoso.	Dado que el usuario está activo y escribe su nombre de usuario y su contraseña correctos en la pantalla de inicio de sesión.	Cuando presiona el botón "Ingresar".	El sistema valida las credenciales, abre el menú principal y muestra en la barra de estado el nombre del usuario y su rol (ADMINISTRADOR o VENDEDOR).
				2	Credenciales incorrectas.	Dado que el usuario escribe un nombre de usuario que no existe o una contraseña incorrecta.	Cuando presiona el botón "Ingresar".	El sistema no permite el ingreso, muestra el mensaje "Usuario o contraseña incorrectos." sin indicar cuál de los dos datos falló, limpia el campo Contraseña y conserva el nombre de usuario.
				3	Usuario desactivado.	Dado que el administrador desactivó la cuenta del usuario y este escribe sus credenciales correctas.	Cuando presiona el botón "Ingresar".	El sistema no permite el ingreso y muestra el mensaje "Usuario o contraseña incorrectos.", igual que con credenciales inválidas.
				4	Tres intentos fallidos seguidos.	Dado que el usuario ya falló dos intentos seguidos de inicio de sesión.	Cuando falla un tercer intento.	El sistema deshabilita el botón "Ingresar" durante 1 minuto y muestra el mensaje "Demasiados intentos fallidos. Intente de nuevo en 1 minuto.".
				5	Campos vacíos.	Dado que el usuario deja vacío el campo Usuario, el campo Contraseña o ambos.	Cuando presiona el botón "Ingresar".	El sistema no envía la solicitud al servidor, resalta los campos vacíos y muestra el mensaje "Debe ingresar usuario y contraseña.".
HU 2	Como un usuario (vendedor o administrador)	Necesito cerrar mi sesión al terminar de trabajar	Con la finalidad de que otra persona no use mi cuenta en la misma caja	1	Cierre de sesión.	Dado que el usuario tiene una sesión abierta y no tiene ninguna factura en proceso.	Cuando selecciona la opción "Cerrar sesión" del menú principal.	El sistema cierra la sesión, libera la conexión con el servidor y muestra la pantalla de inicio de sesión con los campos vacíos.
				2	Factura sin emitir.	Dado que el vendedor tiene una factura con líneas	Cuando selecciona la opción "Cerrar	El sistema pregunta "Tiene una factura sin emitir. ¿Desea

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
						agregadas que todavía no ha emitido.	sesión".	descartarla y cerrar sesión?". Si responde "Si", descarta la factura y cierra la sesión; si responde "No", vuelve a la factura sin cambios.
HU 3	Como un administrador	Necesito crear, consultar, modificar y eliminar las cuentas de vendedores y administradores	Con la finalidad de controlar quién tiene acceso al sistema	1	Creación exitosa de un usuario.	Dado que el administrador completa nombre completo, teléfono, correo, nombre de usuario, contraseña, confirmación de la contraseña y rol. La contraseña tiene al menos 8 caracteres.	Cuando presiona el botón "Guardar".	El sistema crea el usuario con un ID automático y estado Activo, muestra el mensaje "Usuario creado correctamente." y lo agrega a la tabla de usuarios.
				2	Nombre de usuario ya registrado.	Dado que el nombre de usuario ingresado ya pertenece a otra cuenta.	Cuando presiona el botón "Guardar".	El sistema no crea la cuenta, muestra el mensaje "El nombre de usuario ya existe.", resalta el campo Usuario y conserva los demás datos ingresados.
				3	Contraseña inválida.	Dado que la contraseña tiene menos de 8 caracteres o no coincide con el campo "Confirmar".	Cuando presiona el botón "Guardar".	El sistema no guarda el usuario y muestra el mensaje "La contraseña debe tener al menos 8 caracteres y coincidir con la confirmación.".
				4	Eliminar un usuario.	Dado que el administrador selecciona en la tabla un usuario que no ha emitido facturas.	Cuando presiona el botón "Eliminar" y confirma la acción.	El sistema borra el usuario de la base de datos, muestra el mensaje "Usuario eliminado correctamente." y lo quita de la tabla.
				5	Usuario con facturas.	Dado que el administrador selecciona un usuario que ya emitió facturas.	Cuando presiona el botón "Eliminar".	El sistema no lo borra, muestra el mensaje "El usuario tiene facturas registradas; se desactivará para conservar el historial." y, si el administrador confirma, lo deja Inactivo: ya no puede iniciar sesión.
				6	Acceso de un vendedor.	Dado que inició sesión un usuario con rol VENDEDOR.	Cuando revisa las opciones del menú principal.	El sistema no muestra la opción "Usuarios", de modo que el vendedor no puede entrar a este módulo.
HU 4	Como un vendedor	Necesito registrar a un cliente con sus	Con la finalidad de poder emitirle	1	Registro exitoso con datos válidos.	Dado que el vendedor completa tipo de	Cuando presiona el botón "Guardar".	El sistema valida la información, crea el cliente con un ID

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
		datos de identificación y de contacto	facturas			identificación, número de identificación, nombre o razón social, teléfono, correo y dirección. Tipo, identificación y nombre son obligatorios. La cédula física tiene formato X-XXXX-XXXX y la jurídica X-XXX-XXXXXX.		automático y la fecha de registro del día, muestra el mensaje "Cliente registrado correctamente." y lo agrega a la tabla de clientes.
				2	Identificación ya registrada.	Dado que el número de identificación ingresado ya pertenece a otro cliente.	Cuando presiona el botón "Guardar".	El sistema no crea el cliente, muestra el mensaje "Ya existe un cliente con la identificación indicada.", resalta el campo Identificación y conserva los demás datos.
				3	Formato de identificación inválido.	Dado que el número de identificación no cumple el formato del tipo seleccionado (por ejemplo, una cédula física sin guiones).	Cuando presiona el botón "Guardar".	El sistema muestra el mensaje "Formato de identificación inválido para el tipo seleccionado.", marca el campo como inválido y no guarda el cliente.
				4	Formato de correo electrónico inválido.	Dado que el vendedor ingresa un correo que no cumple el formato usuario@dominio (por ejemplo, compras.laesquina.cr).	Cuando presiona el botón "Guardar".	El sistema muestra el mensaje "Formato de correo electrónico inválido.", marca el campo Correo como inválido y no guarda el cliente.
				5	Campos obligatorios incompletos.	Dado que el vendedor deja vacío el tipo de identificación, la identificación o el nombre.	Cuando presiona el botón "Guardar".	El sistema no crea el cliente, resalta los campos incompletos y muestra el mensaje "Debe completar los campos obligatorios.".
HU 5	Como un vendedor	Necesito buscar a un cliente y actualizar sus datos de contacto	Con la finalidad de mantener su información al día	1	Búsqueda por coincidencia parcial.	Dado que el vendedor escribe parte del nombre o de la identificación de un cliente (por ejemplo, "esquina").	Cuando presiona el botón "Buscar".	El sistema muestra en la tabla solo los clientes cuyo nombre o identificación contienen el texto, sin distinguir mayúsculas de minúsculas.
				2	Búsqueda sin resultados.	Dado que ningún cliente coincide con el texto buscado.	Cuando presiona el botón "Buscar".	El sistema deja la tabla vacía y muestra el mensaje "No se encontraron clientes.".
				3	Edición exitosa.	Dado que el vendedor selecciona un cliente en la tabla, sus datos se cargan en el formulario y modifica el teléfono, el correo o la dirección con información	Cuando presiona el botón "Guardar".	El sistema actualiza el cliente, muestra el mensaje "Cliente actualizado correctamente." y la tabla refleja los nuevos datos.

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
						válida.		
				4	Descartar cambios.	Dado que el vendedor está modificando los datos de un cliente.	Cuando presiona el botón "Limpiar".	El sistema descarta los cambios, deja el formulario vacío y el cliente conserva sus datos originales.
HU 6	Como un administrador	Necesito eliminar un cliente registrado	Con la finalidad de mantener la lista de clientes limpia sin perder el historial de facturas	1	Eliminación confirmada.	Dado que el administrador selecciona en la tabla un cliente que no tiene facturas.	Cuando presiona el botón "Eliminar" y responde "Sí" a la confirmación.	El sistema borra el cliente de la base de datos, muestra el mensaje "Cliente eliminado correctamente." y lo quita de la tabla. Las otras cajas dejan de verlo al actualizar su lista.
				2	Cliente con facturas.	Dado que el administrador selecciona un cliente que ya tiene facturas.	Cuando presiona el botón "Eliminar".	El sistema no lo borra y muestra el mensaje "El cliente tiene facturas registradas; se desactivará para conservar el historial.". Si el administrador confirma, el cliente queda inactivo y deja de aparecer en Nueva factura.
				3	Eliminación cancelada.	Dado que el administrador presionó "Eliminar" sobre un cliente.	Cuando responde "No" a la confirmación.	El sistema no hace ningún cambio.
				4	Intento de un vendedor.	Dado que inició sesión un usuario con rol VENDEDOR y abre la gestión de clientes.	Cuando selecciona un cliente en la tabla.	El sistema mantiene deshabilitado el botón "Eliminar", porque solo el administrador puede usarlo.
HU 7	Como un administrador	Necesito crear, editar y eliminar las categorías de productos (abarrotes, bebidas, limpieza, higiene...)	Con la finalidad de mantener el catálogo ordenado	1	Creación exitosa de una categoría.	Dado que el administrador escribe el nombre y la descripción de una categoría nueva en la pestaña Categorías.	Cuando presiona el botón "Guardar".	El sistema crea la categoría con un ID automático, muestra el mensaje "Categoría creada correctamente." y la agrega a la lista y al filtro de productos.
				2	Nombre de categoría repetido.	Dado que ya existe una categoría con el mismo nombre.	Cuando presiona el botón "Guardar".	El sistema no crea la categoría y muestra el mensaje "Ya existe una categoría con ese nombre.".
				3	Eliminar una categoría con productos.	Dado que el administrador selecciona una categoría que todavía tiene productos.	Cuando presiona el botón "Eliminar".	El sistema no elimina la categoría y muestra el mensaje "No se puede eliminar la categoría porque tiene productos.".
				4	Eliminar una categoría vacía.	Dado que el administrador selecciona una categoría sin	Cuando presiona el botón "Eliminar" y	El sistema elimina la categoría y la quita de la lista y del filtro de

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
						productos.	confirma la acción.	productos.
HU 8	Como un administrador	Necesito registrar, consultar, modificar y eliminar productos indicando su tipo (abarrotes, bebida o artículo del hogar)	Con la finalidad de poder venderlos y controlar sus existencias	1	Registro exitoso de un abarrotes.	Dado que el administrador elige el tipo Abarrotes y completa código, nombre, categoría, valor de compra, valor de venta, existencias, stock mínimo, fecha de vencimiento, peso y si es de canasta básica.	Cuando presiona el botón "Guardar".	El sistema crea el producto con un ID automático, le asigna IVA de 1 % si es de canasta básica o de 13 % si no lo es, muestra el mensaje "Producto registrado correctamente." y lo agrega a la tabla ordenada por nombre.
				2	Cambio del tipo de producto.	Dado que el administrador está llenando el formulario de un producto nuevo.	Cuando cambia el tipo de producto a Bebida.	El sistema cambia el recuadro de datos específicos para pedir volumen (ml), unidades por paquete y si es retornable, e indica que el producto pagará IVA de 13 %.
				3	Código repetido.	Dado que el código ingresado ya pertenece a otro producto.	Cuando presiona el botón "Guardar".	El sistema no guarda el producto, muestra el mensaje "Ya existe un producto con ese código." y resalta el campo Código.
				4	Valores inválidos.	Dado que el valor de venta es 0 o menor que el valor de compra, o las existencias son negativas.	Cuando presiona el botón "Guardar".	El sistema no guarda el producto, resalta el campo con error y muestra el mensaje correspondiente, por ejemplo "El valor de venta debe ser mayor que 0 y no menor que el valor de compra."
				5	Eliminar un producto.	Dado que el administrador selecciona un producto que no aparece en ninguna factura.	Cuando presiona el botón "Eliminar" y confirma.	El sistema borra el producto de la base de datos y de la tabla. Si el producto ya aparece en facturas, no lo borra: lo desactiva para que no se pueda vender y muestra el motivo.
HU 9	Como un administrador	Necesito registrar entradas de mercadería y ajustes de existencias	Con la finalidad de que el inventario del sistema coincida con lo que hay en bodega	1	Entrada de mercadería.	Dado que el administrador selecciona un producto con 18 unidades, abre "Ajustar stock...", elige el motivo Compra y escribe 50.	Cuando presiona el botón "Aplicar" y confirma.	El sistema aumenta las existencias a 68, muestra el mensaje "Existencias actualizadas: 68 unidades." y quita la alerta de stock bajo si ya no aplica.
				2	Ajuste negativo válido.	Dado que el producto tiene 18 unidades y el administrador	Cuando presiona el botón "Aplicar" y	El sistema reduce las existencias a 13 y actualiza la tabla de

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
						registra una merma de 5.	confirma.	productos.
				3	Ajuste que deja existencias negativas.	Dado que el producto tiene 9 unidades y el administrador registra un ajuste de -12.	Cuando presiona el botón "Aplicar".	El sistema no aplica el ajuste y muestra el mensaje "El ajuste dejaría las existencias en negativo.".
				4	Motivo sin indicar.	Dado que el administrador escribe la cantidad pero no elige el motivo del ajuste.	Cuando presiona el botón "Aplicar".	El sistema no aplica el ajuste y muestra el mensaje "Debe indicar el motivo del ajuste.".
HU 10	Como un administrador	Necesito ver qué productos están por debajo del stock mínimo	Con la finalidad de reabastecerlos a tiempo	1	Alerta en el menú principal.	Dado que hay productos cuyas existencias son menores o iguales a su stock mínimo.	Cuando el administrador entra al menú principal.	El sistema muestra el aviso "N productos bajo el stock mínimo" con los nombres de esos productos.
				2	Consulta de productos bajo el mínimo.	Dado que el administrador está en la pantalla de productos e inventario.	Cuando abre la pestaña "Bajo el mínimo".	El sistema lista los productos bajo el mínimo ordenados de menos a más existencias y los marca en rojo.
				3	Producto reabastecido.	Dado que un producto bajo el mínimo recibe una entrada de mercadería que lo deja por encima del mínimo.	Cuando se aplica el ajuste.	El sistema quita el producto del aviso del menú principal y de la pestaña "Bajo el mínimo".
HU 11	Como un vendedor	Necesito crear una factura seleccionando al cliente y los productos que compra	Con la finalidad de registrar la venta y cobrarle el monto correcto	1	Factura emitida correctamente.	Dado que el vendedor selecciona un cliente activo, agrega productos con cantidades que no superan las existencias y elige el método de pago.	Cuando presiona el botón "Emitir factura".	El sistema calcula subtotal, descuento, IVA de cada línea según el tipo de producto y total; asigna el número consecutivo (por ejemplo FC-000123); descuenta las existencias y muestra la vista previa de la factura.
				2	Cantidad mayor a las existencias.	Dado que el vendedor escribe una cantidad mayor a las unidades disponibles del producto.	Cuando presiona el botón "Agregar".	El sistema no agrega la línea y muestra el mensaje "Stock insuficiente de <producto>: hay N unidades.".
				3	Producto agregado dos veces.	Dado que el producto ya está en la factura.	Cuando el vendedor lo agrega de nuevo.	El sistema suma la cantidad a la misma línea (sin pasar de las existencias) y recalcula los totales.
				4	Aplicar un descuento.	Dado que el vendedor escribe un porcentaje de descuento.	Cuando sale del campo Descuento.	Si el valor está entre 0 y 100, el sistema aplica el descuento sobre el subtotal, calcula el IVA sobre el monto con descuento y actualiza el total; si no, muestra

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
								"El descuento debe estar entre 0 y 100.".
				5	Factura incompleta.	Dado que la factura no tiene cliente seleccionado o no tiene ningún producto.	Cuando presiona el botón "Emitir factura".	El sistema no emite la factura y muestra el mensaje "Debe seleccionar un cliente y agregar al menos un producto.".
HU 12	Como un administrador	Necesito que varias cajas puedan facturar al mismo tiempo	Con la finalidad de atender a más clientes sin vender más producto del que hay en inventario	1	Dos cajas con existencias suficientes.	Dado que dos cajas facturan el mismo producto y la suma de las cantidades no supera las existencias.	Cuando ambas emiten su factura casi al mismo tiempo.	El sistema emite las dos facturas con números consecutivos distintos y descuenta de las existencias la suma de ambas cantidades.
				2	Dos cajas compiten por las últimas unidades.	Dado que quedan 9 unidades de un producto y cada caja tiene una factura con esas 9 unidades.	Cuando ambas emiten su factura al mismo tiempo.	El sistema emite solo una de las facturas; la otra caja recibe el mensaje "Stock insuficiente de <producto>: hay 0 unidades." y conserva su factura para corregirla. Las existencias nunca quedan en negativo.
				3	Venta rechazada.	Dado que una factura fue rechazada por falta de existencias.	Cuando se emite la siguiente factura.	El sistema le asigna el siguiente número consecutivo, sin dejar huecos por la venta rechazada.
HU 13	Como un vendedor	Necesito obtener la factura en un archivo con el desglose de pago	Con la finalidad de entregarla o imprimirla al cliente	1	Generar el archivo de la factura.	Dado que la factura fue emitida y su vista previa está abierta.	Cuando el vendedor presiona "Guardar archivo (.txt)" y elige una carpeta.	El sistema crea el archivo FC-000123.txt con los datos de Fidecompro, número y fecha, vendedor, cliente, cada línea (cantidad, descripción, precio unitario y total), subtotal, descuento, IVA, total y método de pago, y muestra el mensaje "Archivo guardado en <ruta>.".
				2	Error al guardar el archivo.	Dado que la carpeta elegida no tiene permisos de escritura o el disco está lleno.	Cuando el vendedor presiona "Guardar archivo (.txt)".	El sistema muestra el mensaje "No se pudo guardar el archivo de la factura."; la factura sigue emitida y el vendedor puede intentarlo en otra carpeta.
				3	Regenerar desde el historial.	Dado que la factura se emitió en otro momento y aparece en el historial.	Cuando el usuario la selecciona y presiona "Regenerar archivo".	El sistema genera el archivo con el mismo contenido de la factura original, incluidos los precios del momento de la venta.

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
HU 14	Como un usuario (vendedor o administrador)	Necesito consultar las facturas por rango de fechas, cliente y estado	Con la finalidad de dar seguimiento a las ventas	1	Consulta por rango de fechas.	Dado que el usuario indica una fecha inicial y una fecha final válidas.	Cuando presiona el botón "Consultar".	El sistema lista las facturas del rango con número, fecha, cliente, vendedor, total y estado, y muestra el total vendido del periodo sumando solo las facturas emitidas.
				2	Filtro por cliente y estado.	Dado que el usuario además elige un cliente y el estado ANULADA.	Cuando presiona el botón "Consultar".	El sistema muestra solo las facturas anuladas de ese cliente dentro del rango de fechas.
				3	Rango de fechas inválido.	Dado que la fecha inicial es posterior a la fecha final.	Cuando presiona el botón "Consultar".	El sistema no realiza la consulta y muestra el mensaje "La fecha inicial no puede ser posterior a la final.".
				4	Consulta sin resultados.	Dado que no hay facturas que cumplan los filtros.	Cuando presiona el botón "Consultar".	El sistema deja la tabla vacía y muestra el mensaje "No hay facturas para los filtros seleccionados.".
HU 15	Como un administrador	Necesito anular una factura emitida por error	Con la finalidad de corregir las ventas y devolver el producto al inventario	1	Anulación exitosa.	Dado que el administrador selecciona en el historial una factura en estado EMITIDA.	Cuando presiona el botón "Anular factura" y confirma la acción.	El sistema cambia el estado a ANULADA, devuelve al inventario las unidades de cada línea y recalcula el total del periodo sin esa factura.
				2	Factura ya anulada.	Dado que la factura seleccionada ya está en estado ANULADA.	Cuando el administrador presiona el botón "Anular factura".	El sistema no hace cambios y muestra el mensaje "La factura ya estaba anulada.".
				3	Intento de un vendedor.	Dado que inició sesión un usuario con rol VENDEDOR y abre el historial de facturas.	Cuando selecciona una factura.	El sistema no muestra el botón "Anular factura", porque solo el administrador puede anular.
HU 16	Como un administrador	Necesito una ventana del servidor donde pueda iniciarlo, detenerlo y ver los clientes conectados y las operaciones que atiende	Con la finalidad de comprobar que el servidor atiende a varias cajas al mismo tiempo y que todo queda guardado en la base de datos	1	Iniciar el servidor.	Dado que la ventana del servidor está abierta, el puerto es 5000 y la base de datos está disponible.	Cuando el administrador presiona "Iniciar".	El servidor abre el puerto, muestra el estado "En ejecución" y queda esperando clientes.
				2	Base de datos no disponible.	Dado que la base de datos no responde.	Cuando el administrador presiona "Iniciar".	El servidor no se inicia y muestra el mensaje "No se pudo conectar con la base de datos.".
				3	Varios clientes conectados.	Dado que el servidor está en ejecución.	Cuando se abren tres ventanas cliente e inician sesión.	La ventana del servidor muestra las tres conexiones, cada una con su hilo, equipo, usuario y hora de conexión. Al cerrar una ventana cliente, su fila

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
								desaparece.
				4	Bitácora de operaciones.	Dado que hay clientes conectados.	Cuando un cliente crea, consulta, modifica o elimina un registro.	La bitácora agrega una fila con la hora, el hilo, la operación y su resultado (OK o el error).
				5	Detener el servidor.	Dado que el servidor está en ejecución con clientes conectados.	Cuando el administrador presiona "Detener" y confirma.	El servidor cierra las conexiones y el puerto; las ventanas cliente muestran "Se perdió la conexión con el servidor."

Resumen de las historias

ID	Historia	Rol	Prioridad	Puntos	Pantalla
HU 1	Iniciar sesión	Todos	Alta	3	P1
HU 2	Cerrar sesión	Todos	Alta	1	P2
HU 3	Administrar usuarios	Administrador	Alta	5	P8
HU 4	Registrar cliente	Vendedor	Alta	3	P3
HU 5	Buscar y editar cliente	Vendedor	Alta	2	P3
HU 6	Eliminar cliente	Administrador	Media	2	P3
HU 7	Administrar categorías	Administrador	Alta	2	P4, P10
HU 8	Registrar producto	Administrador	Alta	3	P4
HU 9	Ajustar inventario	Administrador	Alta	3	P4
HU 10	Alertas de stock bajo	Administrador	Media	2	P2, P4
HU 11	Crear factura	Vendedor	Alta	8	P5
HU 12	Facturación simultánea	Administrador	Alta	5	P5
HU 13	Generar factura física	Vendedor	Alta	3	P6
HU 14	Historial de facturas	Todos	Media	3	P7
HU 15	Anular factura	Administrador	Media	3	P7
HU 16	Ventana del servidor	Administrador	Alta	5	P9

6. Prototipos de interfaz gráfica

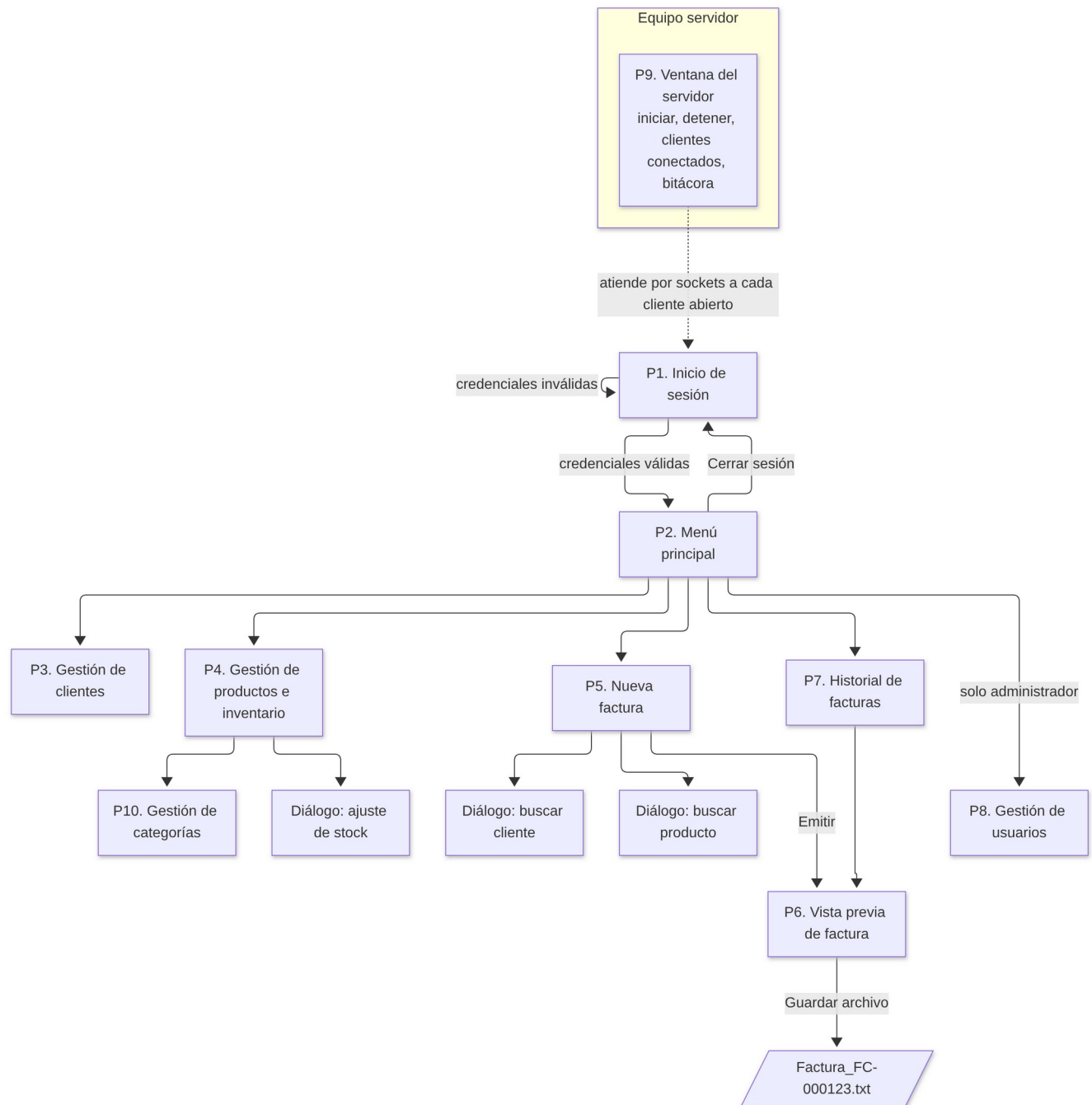
Los prototipos (bocetos o mockups) representan las ventanas JFrame de la aplicación final (look and feel Nimbus): nueve del cliente y una del servidor. Se elaboraron como maquetas HTML/CSS, incluidas en docs/prototipos/html/, y se exportaron a imagen. Los datos mostrados son de ejemplo.

Cada clase que el usuario administra tiene su pantalla para crear, consultar, modificar y eliminar:

Pantalla	Clase que administra	Operaciones
P1 · Inicio de sesión	Usuario	Validar usuario y contraseña
P2 · Menú principal	—	Acceso a todos los módulos según el rol
P3 · Gestión de clientes	Cliente	Crear, consultar, modificar y eliminar
P4 · Productos e inventario	Producto (Abarrote, Bebida,	Crear, consultar, modificar, eliminar y

Pantalla	Clase que administra	Operaciones
	ArticuloHogar)e Inventario	ajustar existencias
P10 · Gestión de categorías	Categoria	Crear, consultar, modificar y eliminar
P8 · Gestión de usuarios	Usuario	Crear, consultar, modificar y eliminar
P5, P6 y P7 · Facturación	Factura y DetalleFactura	Crear, ver la factura física, consultar el historial y anular
P9 · Ventana del servidor	ServidorFacturacion	Iniciar, detener y ver clientes conectados

6.1 Mapa de navegación



Mapa de navegación entre pantallas

6.2 P1 • Inicio de sesión (HU 1)

Campos de usuario y contraseña (oculta), dirección del servidor y mensaje de error ante credenciales inválidas.

Fidecompro · Iniciar sesión

FIDECOMPRO

Sistema de facturación e inventario

Usuario:

mrodriguez

Contraseña:

••••••••

Servidor:

192.168.1.10 : 5000

Usuario o contraseña incorrectos (intento 1 de 3)

Ingresar

Salir

Estado: conectado al servidor

v1.0

PI Inicio de sesión

6.3 P2 · Menú principal (HU 2, HU 10)

Acceso a todos los módulos mediante menú y botones, resumen del día y alerta de productos bajo el stock mínimo. La barra de estado muestra el usuario, su rol y el estado de la conexión. El botón *Usuarios* solo aparece para administradores.



P2 Menú principal

6.4 P3 · Gestión de clientes (HU 4, HU 5, HU 6)

CRUD completo de clientes: formulario para crear y modificar, búsqueda, tabla de clientes y botón *Eliminar*. El botón *Actualizar lista* vuelve a pedir los datos al servidor para ver los cambios hechos desde otras cajas.

Fidecompro · Gestión de clientes

Datos del cliente

Tipo de identificación: Jurídica

Identificación: 3-101-456789

Nombre / Razón social: Abastecedor La Esquina S.A.

Teléfono: 2222-3344

Correo: compras@laesquina.cr

Dirección: Heredia, San Pablo, 200 m norte

Activo: ☒ (desmarcar para ocultarlo al facturar)

Nuevo

Guardar

Limpiar

Eliminar

Actualizar lista

Buscar: esquina

Buscar

Identificación	Nombre	Teléfono	Correo	Registro
3-101-456789	Abastecedor La Esquina S.A.	2222-3344	compras@laesquina.cr	12/09/2026
1-1234-0567	Pulpería Doña Ana	8888-1020	ana.pulperia@gmail.com	20/09/2026
3-102-778899	Minisúper El Ahorro Ltda.	2560-7788	admin@elahorro.cr	01/10/2026

3 clientes encontrados

Usuario: María Rodríguez

P3 Gestión de clientes

6.5 P4 · Productos e inventario (HU 8, HU 9, HU 10)

Pestañas para productos, categorías y productos bajo el mínimo. El formulario cambia según el tipo de producto elegido (abarrotes, bebida o artículo del hogar). Los productos bajo el mínimo se resaltan en rojo. Los botones permiten crear, modificar, eliminar y ajustar el stock.

Fidecompro · Productos e inventario

Productos

Categorías

Bajo el mínimo (4)

Datos del producto

Tipo de producto: Abarrote

Categoría: Abarrotes

Código: ABR-0012

Nombre: Arroz 99% grano entero 2 kg

Valor compra (C): 1 520,00

Valor venta (C): 1 950,00

Existencias: 18 unidades

Stock mínimo: 25

Datos de abarrote

Fecha vencimiento: 15/03/2027

Peso (kg): 2,0

☒ Canasta básica (IVA 1 %)

Este recuadro cambia según el tipo: Bebida (volumen, unidades por paquete, retornable) · Artículo del hogar (marca, presentación, advertencias)

Nuevo

Guardar

Ajustar stock...

Eliminar

Actualizar lista

Filtrar por categoría: Todas

buscar por código o nombre

Buscar

Código	Nombre	Tipo	Categoría	Precio	Stock	Estado
ABR-0012	Arroz 99% grano entero 2 kg	Abarrote	Abarrotes	€1 950	18	Bajo mínimo
LIM-0230	Detergente en polvo 3 kg	Art. hogar	Limpieza	€5 400	9	Bajo mínimo
HIG-0045	Papel higiénico 24 rollos	Art. hogar	Higiene	€8 750	120	Disponible
BEB-0101	Refresco cola 3 L (caja 6)	Bebida	Bebidas	€9 300	64	Disponible

Ordenado por nombre (Comparable) · pestaña "Bajo el mínimo" ordena por existencias (Comparator)

Usuario: María Rodríguez

P4 Productos e inventario

6.6 P5 · Nueva factura (HU 11, HU 12)

Selección de cliente, agregado de productos con validación de existencias, tabla de líneas con el IVA de cada tipo de producto, método de pago, descuento y totales calculados.

Fidecompro · Nueva factura

Ciente

Identificación:
3-101-456789

Buscar cliente...

Abastecedor La Esquina S.A. · 2222-3344

Agregar producto

Código:
BEB-0101

Buscar...

Cantidad:
5

Disponible: 64

Agregar →

#	Código	Descripción	Cant.	Precio unit.	IVA	Total línea
1	ABR-0012	Arroz 99% grano entero 2 kg	10	₡1 950,00	1%	₡19 695,00
2	HIG-0045	Papel higiénico 24 rollos	4	₡8 750,00	13%	₡39 550,00
3	BEB-0101	Refresco cola 3 L (caja 6)	5	₡9 300,00	13%	₡52 545,00

Método de pago:

Transferencia

Descuento (%):

2

Quitar línea

Cancelar

Subtotal

₡101 000,00

Descuento (2%)

-₡2 020,00

Impuesto (IVA)

₡10 574,20

TOTAL

₡109 554,20

Emitir factura

Vendedor: Carlos Jiménez

06/10/2026 10:42

P5 Nueva factura

6.7 P6 · Vista previa de la factura física (HU 13)

Contenido exacto del archivo que se genera, con el desglose de pago, y botones para guardarlo como .txt o .html.

Fidecompro · Factura FC-000123

=====

FIDECOMPRO - VENTAS AL POR MAYOR
Cédula jurídica 3-101-000000 Tel 2000-0000

=====

Factura N.º : FC-000123 Fecha: 06/10/2026 10:43
Vendedor : Carlos Jiménez
Cliente : Abastecedor La Esquina S.A.
Identific. : 3-101-456789 Tel : 2222-3344

Cant	Descripción	P.Unit	Total
10	Arroz 99% grano entero 2 kg	1 950,00	19 500,00
4	Papel higiénico 24 rollos	8 750,00	35 000,00
5	Refresco cola 3 L (caja 6)	9 300,00	46 500,00

Subtotal :

101 000,00

Descuento 2% :

-2 020,00

IVA :

10 574,20

TOTAL :

₡ 109 554,20

Método de pago: TRANSFERENCIA

=====

¡Gracias por su compra!

Guardar archivo (.txt)

Guardar como HTML

Cerrar

Archivo: facturas/FC-000123.txt

Estado: EMITIDA

P6 Vista previa de factura

6.8 P7 · Historial de facturas (HU 14, HU 15)

Filtros por fecha, cliente y estado; total del periodo y acciones para ver el detalle, regenerar el archivo o anular.

Fidecompro · Historial de facturas

Filtros

Desde: 01/10/2026 Hasta: 06/10/2026 Cliente: (todos) Estado: Todas Consultar

Número	Fecha	Cliente	Vendedor	Total	Estado
FC-000123	06/10/2026 10:43	Abastecedor La Esquina S.A.	Carlos Jiménez	€109 554,20	EMITIDA
FC-000122	06/10/2026 09:15	Pulpería Doña Ana	Carlos Jiménez	€34 870,00	EMITIDA
FC-000121	05/10/2026 16:02	Minisúper El Ahorro Ltda.	María Rodríguez	€212 400,00	ANULADA
FC-000120	05/10/2026 11:30	Abastecedor La Esquina S.A.	Carlos Jiménez	€88 015,50	EMITIDA

Total del periodo (emitidas): €232 439,70
Ver detalle
Regenerar archivo
Anular factura

4 facturas
Usuario: María Rodríguez

P7 Historial de facturas

6.9 P8 · Gestión de usuarios (HU 3)

Exclusiva del administrador: creación, consulta, modificación (incluido el rol y el estado activo), eliminación y restablecimiento de contraseña de usuarios.

Fidecompro · Gestión de usuarios (Administrador)

Datos del usuario

Nombre completo: Carlos Jiménez Mora Usuario: cjimenez

Contraseña: Confirmar:

Rol: VENDEDOR Activo: ☒

Nuevo Guardar Restablecer contraseña Eliminar Actualizar lista

Usuario	Nombre	Rol	Estado	Último ingreso
mrodriguez	María Rodríguez Solís	ADMINISTRADOR	Activo	06/10/2026 08:01
cjimenez	Carlos Jiménez Mora	VENDEDOR	Activo	06/10/2026 08:20
lvargas	Laura Vargas Pérez	VENDEDOR	Inactivo	14/09/2026 17:45

Las contraseñas se guardan cifradas (SHA-256)
Usuario: María Rodríguez

P8 Gestión de usuarios

6.10 P9 · Ventana del servidor (HU 16)

Se ejecuta en el equipo servidor, aparte de las cajas. Permite elegir el puerto, iniciar y detener el servidor, y muestra los clientes conectados (cada uno con su hilo) y una bitácora con cada operación

que llega y su resultado. Es la pantalla que demuestra en el video que el servidor atiende a varios clientes a la vez.

Servidor Fidecompro

Servidor

Puerto:5000Base de datos:MySQL · fidecomproIniciarDetener

Estado:● En ejecución desde 08:00:12Clientes conectados: 3Hilos activos: 3 de 10

Cientes conectados

Hilo	Equipo (IP)	Usuario	Rol	Conectado desde
Hilo-1	192.168.1.21 (Caja 1)	cjimenez	VENDEDOR	08:02:40
Hilo-2	192.168.1.22 (Caja 2)	apineda	VENDEDOR	08:05:11
Hilo-3	192.168.1.10 (Administración)	mrodriguez	ADMINISTRADOR	08:01:03

Bitácora de operaciones

Hora	Hilo	Operación	Resultado
08:41:05	Hilo-1	CREAR_FACTURA FC-000123 (cliente 3-101-456789)	OK
08:41:05	Hilo-2	CREAR_FACTURA (ABR-0012 × 9)	Stock insuficiente
08:39:47	Hilo-3	ACTUALIZAR_PRODUCTO LIM-0230	OK
08:38:20	Hilo-3	ELIMINAR_CLIENTE 1-0987-0654	OK
08:36:02	Hilo-2	CREAR_CLIENTE 3-102-778899	OK

Cada cliente conectado se atiende en su propio hilo

Datos guardados en la base de datos

P9 Ventana del servidor

6.11 P10 • Gestión de categorías (HU 7)

Se abre desde la pantalla de productos. CRUD de categorías con su nombre y descripción; la tabla indica cuántos productos tiene cada una, porque una categoría con productos no se puede eliminar.

Fidecompro · Gestión de categorías (Administrador)

Datos de la categoría

ID:4

Nombre:Higiene

Descripción:Papel higiénico, jabón, pasta dental

Nuevo

Guardar

Eliminar

Actualizar lista

ID	Nombre	Descripción	Productos
1	Abarrotes	Arroz, frijoles, aceite, café, azúcar	42
2	Bebidas	Refrescos, jugos y agua	18
3	Limpieza	Detergentes, cloro y desinfectantes	15
4	Higiene	Papel higiénico, jabón, pasta dental	11

Una categoría que todavía tiene productos no se puede eliminar.

4 categorías

Usuario: María Rodríguez

P10 Gestión de categorías

7. Trazabilidad historias – clases – pantallas

Historia	Clases principales	Pantalla
HU 1, HU 2	Usuario, RegistroUsuarios, CredencialesInvalidasException, VentanaLogin	P1, P2
HU 3	Usuario, Rol, RegistroUsuarios	P8
HU 4 a HU 6	Persona, Cliente, RegistroClientes, RegistroNoEncontradoException	P3
HU 7, HU 8	Inventario, Categoria, Producto, Abarrote, Bebida, ArticuloHogar	P4, P10
HU 9, HU 10	Producto, Inventario, ComparadorPorExistencias	P2, P4
HU 11, HU 12	Factura, DetalleFactura, Inventario, StockInsuficienteException (y en la arquitectura prevista: ServicioFacturacion, ManejadorCliente)	P5
HU 13	Factura, DetalleFactura, Mostrable	P6
HU 14, HU 15	Factura, EstadoFactura, RegistroFacturas	P7
HU 16	VentanaServidor, ServidorFacturacion, ManejadorCliente	P9

8. Plan de demostración del proyecto final

El video final debe mostrar todas las funcionalidades, el CRUD completo desde las pantallas, que todo queda guardado en la base de datos y que el servidor soporta varios clientes a la vez. Esta matriz muestra qué operación del CRUD tiene cada entidad y en qué pantalla se hace:

Entidad	Crear	Consultar	Modificar	Eliminar	Pantalla
Usuarios	Sí	Sí	Sí	Sí (si tiene facturas, se desactiva)	P8
Clientes	Sí	Sí	Sí	Sí (si tiene facturas, se desactiva)	P3
Categorías	Sí	Sí	Sí	Sí (si no tiene productos)	P10
Productos	Sí	Sí	Sí	Sí (si está en facturas, se desactiva)	P4
Facturas	Sí	Sí (historial)	No: una factura emitida no se modifica	Se anula, no se borra	P5, P6, P7

Guion previsto para el video:

1. Abrir la ventana del servidor (P9), iniciarlo y mostrar que se conecta a la base de datos.
2. Abrir dos o tres ventanas cliente al mismo tiempo e iniciar sesión con usuarios distintos; mostrar en P9 que cada uno tiene su hilo.
3. Hacer el CRUD de usuarios, clientes, categorías y productos: crear, buscar, modificar y eliminar un registro de cada uno.
4. Después de cada operación, mostrar la tabla correspondiente en la base de datos (MySQL Workbench o la consola de Derby) para comprobar que quedó guardada.
5. Desde otra caja, presionar *Actualizar lista* y mostrar que ve los cambios hechos por la primera.
6. Emitir una factura, ver la vista previa y generar el archivo de la factura física.
7. Facturar el mismo producto desde dos cajas casi al mismo tiempo: una factura sale y la otra recibe el aviso de stock insuficiente, sin existencias negativas.
8. Consultar el historial, anular una factura y mostrar que las existencias vuelven al inventario.
9. Cerrar todo, volver a abrir el servidor y un cliente, y mostrar que los datos siguen ahí.